ÓBUDAI EGYETEM
ÓBUDA UNIVERSITY

NEUMANN JÁNOS
INFORMATIKAI KAR

Biztonságosabb titkosítási kulcs használat kis teljesítményű Linux segédeszközökkel

OE-NIK
2020

Szerzők: **Kuklin István Alexander**
Vajnai Viktor

Konzulensek: **Ivicsics Sándor**
Vezető fejlesztő, Microsec Zrt.
Lovas István
Tanársegéd, Óbudai Egyetem

Tartalomjegyzék

1. Modern rejtjelezés a civil szférában **4**
1.1. Magánjellegű, bizalmas üzenetváltás az egyén szintjén? 4
1.1.1. Megengedhető tehát ennek a technológiának a használata? 5
1.2. Titkosítás a digitális térben 5
1.3. Lehetőség vagy önámítás? 6
1.3.1. A végponttal szembeni bizalom 7
1.4. A biztonság növelése a felelősségek szétosztása által 9

2. A titkosítás és digitális aláírás alapfogalmai **10**
2.1. Példa szimmetrikus kulcsú titkosításra: Caesar-rejtjel 11
2.2. Általánosabb titkosítási terminológia 12
2.2.1. Szimmetrikus kulcsú titkosítás általánosan 13
2.2.2. Aszimmetrikus titkosítás általánosan 14

3. Hitelesebb digitális aláírásokért **16**
3.1. Hol alkalmazunk digitális aláírásokat? 16
3.2. GnuPG kulcsok védelme 17
3.2.1. GnuPG és OpenSSL 17
3.2.2. A csomagokról és a csomagkezelőről röviden 18
3.3. A végpont elválasztása a kritikus felelősségtől - lehetőségek 18
3.3.1. Megoldás erre a célra kijelölt számítógéppel 19
3.3.2. Megoldás ugyanazzal a számítógéppel 20
3.3.3. Megoldás intelligens kártyával 21
3.3.4. Megoldás erre kijelölt hardverrel és intelligens kártya emulációval . . 23

4. A kulcstároló architektúra **26**

5. Végponti intelligens kártya emuláció megvalósítása **29**
5.1. Nélkülözhetetlen -e a kártya emuláció? 29
5.2. Az illesztő rétegek áttekintése 29
5.3. Lehetséges kommunikációs illesztési szintek emulációhoz 31
5.3.1. Egy létező megvalósítás a PC/SC illesztés szintjén 32
5.4. Emuláció az `scdaemon` program szintjén 33
5.4.1. A GnuPG beszerzése és fordítása 33
5.4.2. Egy Assaun modul működése 34
5.4.3. Egyéni kártyaimplementáció betöltése 36

5.4.4. A könyvtárütközések elkerülése 36
5.5. Látszólagos intelligens kártya implementáció 38
5.5.1. noop implementáció 39
5.5.2. Statikus implementáció 39
5.6. A nyilvános kulcs kommunikálása a `gpg` kliensnek 40
5.6.1. Kulcs ujjlenyomat 40
5.6.2. RSA paraméterek olvasása - OpenPGP Message Format 41
5.6.3. Nyilvános RSA paraméterek felküldése S-expression kifejezések formájában 42

6. Kulcstároló eszköz megvalósítása korai Android eszközön 44
6.1. Kapcsolattartás a kulcstároló eszközzel 44
6.1.1. Kommunikációs protokoll 44
6.2. Függvénykönyvtárak fordítása ARM processzoros Android eszközre 45
6.2.1. Autotools projektek fordítása 46
6.2.2. CMake projektek fordítása 47
6.2.3. A sikeresen lefordított könyvtárak 47
6.2.4. Statikusan és dinamikusan linkelt könyvtárak használata 48
6.3. Egy példa program futtatása az Android parancsértelmezőben 53
6.4. Az elkészült könyvtár használata az alkalmazásban JNI segítségével 54

7. Összefoglalás 56

8. Mellékletek 58
A. Alkalmazásfejlesztő környezet korai Android eszközökhöz 58
F. Forráskód részletek gyűjteménye 59
K. Parancssori kimenetek gyűjteménye 65

Jegyzékek 71
Ábrák jegyzéke 71
Listázások jegyzéke 71
Táblázatok jegyzéke 72

Szójegyzék 74

Rövidítések 78

Hivatkozások 79

1. Modern rejtjelezés a civil szférában

1.1. Magánjellegű, bizalmas üzenetváltás az egyén szintjén?

A titkosított digitális kommunikáció hatósági szabályozása gyakran vet fel kérdéseket politikai és magánéleti körökben is. A problémakör megoldása nem triviális, hiszen végső soron az erre adott válaszok határozzák meg az egyén magánjellegű információcseréhez fűződő jogát, illetve az igazságszolgáltatás által alkalmazható eszközök hatókörét. A digitális eszközök rohamos térnyerésének napjaiban, a felmerülő kérdések jelentőségét az is érzékelteti, hogy a „magánélethez való jog" (az angol nyelvű fordításokban: „right to privacy") kifejezés már több mint 30 állam alkotmányában szerepel. [6]

Gyakran ismételt érv a hatóságok fentiekre vonatkozó hatalmának kiterjesztése mellett a hírszerzési eszközök által szavatolt nagyobb társadalmi biztonság, a hatékonyabb és gyorsabb fellépés a bűnüldözésben. Bár ez utóbbi kétségkívül előnyös oldala a egyezségnek - és az emberek nem elhanyagolható nagyságú körében támogatott gondolat [1] -, mégis, egy ilyen intézmény aránytalanul nagy befolyást is adhat egy állam mindenkori kormányzatának, igen rossz esetben akár egy más jellegű fenyegetettséget megteremtve. Noha a társadalom jelentős része tehát üdvözölné az egyéni magánszféra terhére realizált nagyobb védettséget, a személyes adatokkal való kormányzati visszaélések lehetősége mégis sokakat feszélyez, olykor kisebb-nagyobb megmozdulásokat is kiváltva (pl. Snowden-ügy [16]).

Amennyiben ez a szempont nem is feltétlenül élvezne egy adott közösségben jelentősen kiemelkedő prioritást - azaz, utóbbi tagjai, bizalmat szavazva a mindenkori vezetésnek, részben vagy egészben lemondanak az ehhez fűződő jogaikról a vélt nagyobb biztonság javára -, a kérdések nehézsége sajnos nem csak a közvetlen politikai hatalom korlátozásában rejlik.

Adatbányászat - miért van, és hogyan jelenthet jelen esetben problémát?

Napjainkban, talán még a kormányzati szerveket is felülmúlóan releváns, digitális magánszférán leépítésére törekvő szereplő formájában azok a vállalatok jelennek meg, akik szokásainkról képet alkotva, célzott hirdetéseket juttatnak a képernyőinkre; illetve azon társaságok, akiktől a profil alkotáshoz szükséges információkat az előbbiek felvásárolják. A továbbított adatok jellegükben szerteágazóak lehetnek - szerepeljen itt néhány példa, a teljesség igénye nélkül: melyik weboldalról érkeztünk; milyen stílusban, milyen témákat érintve folytatunk párbeszédet elektronikus üzenetváltásainkban; milyen alkalmazásokat telepítünk telefonunkra; a nap melyik időszakában használjuk melyik elektronikus eszközünket, hol tartózkodunk, kikkel milyen kapcsolatban állunk, stb. - gyakorlatilag bármilyen, személyünkre jellemző

információ eladható lehet. [17]

Miért van erre szükség? Véleményünk szerint részben azért, mert az a vállalatok tapasztalata, hogy az internetes szolgáltatásokat (az internetes kapcsolaton felül) a felhasználók nem szívesen térítenék meg külön, annak ellenére, hogy ezek biztosítása a vállalatoknak költséget termel. Alternatívaként a társaság eladhatja a célzott hirdetéseket célbajuttatását segítő adatainkat, amit, mint közösség, többé-kevésbé elfogadtunk.[1]

Még ha a fentiek tekintetében le is mondunk a magánszféránkról, az imént ismertetett törekvések sajnos szintén akadályt jelenthetnek, ha alkalmanként ténylegesen szükségünk lenne egy biztonságos kommunikációs csatornára. Néhány további, az utóbbi megteremtését akadályozó tényezőről az 1.3.1. pontban számolunk be.

1.1.1. Megengedhető tehát ennek a technológiának a használata?

Ahogyan láthatjuk, ezek olyan kérdések, amik akár hevesebb viták, és társadalmi feszültség gerjesztésére is alkalmasak lehetnek. A bevezetőben igyekeztünk objektíven, mindkét oldalról megközelíteni a problémát. Ennek az írásnak nem célja, és nem is témaköre, hogy állást foglaljon abban a kérdésben, hogy a magánembernek a közösség milyen mértékben engedheti meg a digitális rejtőzködést.

Feltételezve azonban, hogy a mindenkori politikai vezetés a társadalom bizalmát élvezi, az egyén által alkalmazható titkosításnak, mint technológiának, még mindig hasznát vehetjük, a hivatalos, akár kormányzati, akár vállalati kommunikáció hitelesítésében, vagy elrejtésében (például ipari kémkedés megelőzésében).

A digitális titkosítás modern megvalósítása számos izgalmas, gyakorlati kihívást rejt magában. A matematikai és számítástechnikai technológia a rendelkezésünkre áll, és úgy véljük, hogy az alkalmazásukat akadályozó legnagyobb problémát jelenleg a körülményesen kivitelezhető óvintézkedések jelentik. Ebben a dolgozatban kísérletet teszünk rá, hogy a felmerülő problémák közül egyre - ha csak elméleti tekintetben is, de - az egyén számára is viszonylag könnyen alkalmazható, jobb megoldást adjunk.

1.2. Titkosítás a digitális térben

Személyes érintkezéskor biztonsággal elfogadjuk, hogy az információcsere közvetlenül és pontosan azzal a személlyel valósul meg, akinek az üzeneteinket szánjuk. Ennek közvetlen kö-

[1] Bár ez a probléma szigorúan már nem része a dolgozat témakörének, erről azért értekezünk mégis részletesen, mert nem szerettünk volna túlságosan egyszínű képet festeni a rendkívül árnyalt problémáról.

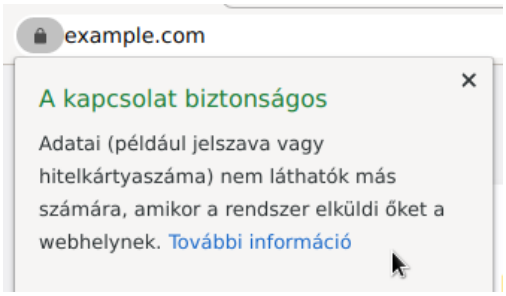
vetkezményeként abban is biztosak lehetünk, hogy:

- az üzeneteket csak a címzett kapja meg (bizalmasság),
- az üzeneteket a címzett eredeti formájában kapja meg (sértetlenség).

Az előbbi feltételek biztosítása digitális eszközökkel azonban korántsem ugyanazokat a kézzelfogható problémákat veti fel. Míg egy bizalmas személyes megbeszélés alkalmával többnyire elegendő, ha az utca egy kevésbé forgalmas szegletén cserélünk információt, addig a távközlési hálózaton minden köztes szereplő a bizalmunkat kell, hogy élvezze. A titkosítás egy olyan kompromisszumos megoldás, amivel elérhetjük, hogy a köztes átjátszóállomások vizsgálata elhanyagolható legyen.

1.3. Lehetőség vagy önámítás?

A modern titkosítás és digitális aláírás eszközeit valószínűleg akkor is alkalmazhatta közvetetten az Olvasó, amikor ezen dokumentumhoz hozzájutott. Ez azért lehetséges, mert a legtöbb böngésző programban ennek sokféle implementációja már beépítetten megtalálható (1.1. ábra).

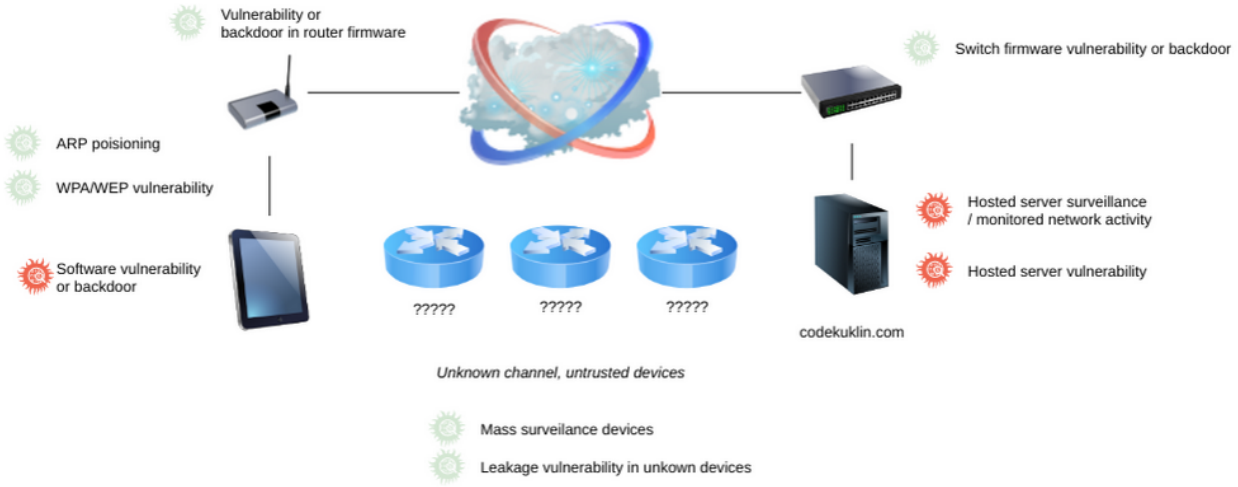


1.1. ábra. A böngésző jelzi a titkosítást

Ez a funkció a köztünk és a webhely távoli kiszolgálója között lévő kommunikációs csatornát védi. Ennek során közel lehetetlenné teszi illetéktelen személyek számára az információ menet közbeni kiszivárogtatását vagy megmásítását.

A titkosítás alkalmazásakor az egyik legnagyobb kihívás abban testesül meg, hogyha a védelem bármely ponton meghiúsul, akkor a célunkat sajnos nem értük el vele. Tetézi a helyzet bonyolultságát, hogy magáról az információ kompromittálódásáról nem kapunk értesítést, legfeljebb a biztonsági rést észlelhetjük a rendszerben - sok esetben a kár bekövetkezése után.

A hálózaton utazó csomagok számtalan hibának és támadási lehetőségnek vannak útközben kitéve.



1.2. ábra. Az információ sebezhetősége útközben

A böngésző által szolgáltatott technológia egy ilyen kapcsolatban a 1.2. ábrán látható sebezhetőségek közül a halványzölddel jelölteket lényegében ártalmatlanítja. A problémát itt a csomagok által bejárt útvonalon lévő eszközök jelentették. Feltéve, hogy a titkosítást megvalósító matematika tényleg megbízható, a legkönnyebben támadható szereplő - a végpont - biztosítását még nem tekintjük teljesen megoldottnak.

1.3.1. A végponttal szembeni bizalom

A böngészőnk által nyújtott végponttól végpontig tartó rejtjelezési lehetőségeknek kevés hasznát vehetjük, ha a számítógépen futó operációs rendszer egészen véletlenül naplózná a hálózati forgalmunkat és a billentyűleütéseinket. Ez történhet hibából vagy vírusfertőzésből adódóan is. Sajnos napjainkban az sem teljesen kizárható, hogy a rendszer felhasználói élményt javító telemetriai eszközei olyan információkat is továbbítanának, amiket kifejezetten nem szeretnénk megosztani a szoftverek gyártóival.

Ha el tudnánk érni azt, hogy a végpontunk tényleg csak azokat a műveleteket végezze el, amik a rejtjelezés megvalósításához szükségesek, akkor ezáltal az útvonal lehallgatóinak a feladatát rendkívül megnehezíthetnénk. Ez azonban a részünkről egyáltalán nem lenne triviális feladat:

- Készen kapott beágyazott hardvereinken - mint például az okostelefonunkon, útválasztóinkon, egyéb „intelligens” eszközeinken - igen korlátozottak a lehetőségeink a szoftver megismerésére, lecserélésére.

- Nem könnyíti meg a helyzetünket az a tendencia sem, hogy szoftver- és hardvertervező vállalatok nemcsak pénzünkért, hanem ezzel egy időben az adatainkért cserébe is árusítják termékeiket.
  - Napjaink legelterjedtebb, személyi számítógépekre szánt operációs rendszere, a Microsoft Windows utóbbi években megjelent kiadásai az alkalmazott megfigyelő eszközök számát fokozatosan növelte, melyet a Microsoft egyes visszajelzésein felül számos felhasználói megfigyelés is alátámasztott [8] [13] [3]
- A nyílt forráskódú operációs rendszerek és szoftverek önmagában a bizalom problémájára nem feltétlenül adnak teljes körű megoldást. Nyitott kérdés marad, hogy megbízunk -e az adott a forráskódban, azaz a közösség által ellenőrzött programban, illetve a tárgykódban, amit letöltünk. Sajnos nem kizárható, hogy a letöltött, előre lefordított változat nem tartalmaz „egyéb kiegészítő javításokat". [15] [25] [9]
  - Amennyiben az előbbi komplikációt kiküszöbölendő, saját kezűleg fordítottuk le a nyílt forrású programokat - ellenőrizve azok forráskódját - a fordító program még mindig ismeretlen tényezőt jelenthet. Példának okáért, a GCC fordítóból bizonyítottan előállítható olyan módosított változat, ami saját (módosítatlan) forráskódjának fordításakor rosszindulatú működést rejt bele a keletkezett tárgykódba. [28] Hasonló jellegű viselkedését már a Microsoft saját C++ fordítóján is észrevettek, ami a keletkezett tárgykódba telemetriai működés indukálására utaló hívásokat rejtett. [18]

A bizalom dilemmája tehát messze vezet minket. Ne feledjük el azonban, hogy idővel, költséggel és a megfelelő módszerrel bármely csatorna által nyújtott védelem lerombolható. Nem célunk, és elviekben sem gondoljuk kivitelezhetőnek, hogy ilyen támadásokat maradéktalanul lehetetlenné tegyünk. Ehelyett arra törekszünk, hogy a támadás végrehajtását elég nehézzé tegyük ahhoz, hogy ne legyen kifizetődő vele próbálkozni.

Mivel a végpont egy általános célú, mind szoftvereiben, mind hardveres felépítésében változó eszköz, ezért az esélye mindig fennáll annak, hogy a telepített funkciók között rosszindulatú rutinok is bekerülnek a működésébe. Éppen ezért, a mi projektünk nem közvetlenül a végpont védelmére fókuszál, hanem ennek a leggyengébb láncszemnek a jelentőségét igyekszik minimalizálni, céleszköz bevetésével.

1.4. A biztonság növelése a felelősségek szétosztása által

Az általános célú informatikai eszközök körében, mint amilyen a személyi számítógép is, a változó és nehezen ellenőrizhető szoftvertartalom mellett az egyik legnagyobb kihívást az jelenti, hogy az eszköz valamilyen kommunikációs hálózatra csatlakozik és ezáltal - hibájából adódóan vagy szándékosan - képes adatszivárogtatásra.

Megtehetjük azonban azt, hogy ezeken az általános célú eszközön érzékeny információt nem tárolunk, ehelyett erre a célra dedikált hardvert alkalmazunk. Ezzel a módszerrel a végpont kritikus felelősségét kiszervezhetjük egy sokkal könnyebben védhető környezetbe.

Az ideális titkosító hardver csak a saját funkcióját tudja ellátni. Feladata, hogy olyan adatokkal dolgozzon és számoljon, amiket az 1.3.1. pontban körüljárt hibalehetőségek okán nem szeretnénk, ha megjelennének a végponton.

Ahhoz, hogy lássuk, milyen jellegű tehát a védendő adat, betekintéssel folytatjuk az alkalmazott titkosítás fogalmairól - mellőzve a specifikus matematikai megvalósítást.

2. A titkosítás és digitális aláírás alapfogalmai

Bevezetjük az alábbi terminológiát:

- Nyílt üzenet: ez az a bárki számára értelmezhető üzenet, aminek tartalma csak a feladóra és a címzettre tartozik és/vagy aminek tartalmi eredetisége, módosítatlansága kritikus.
- Rejtjelezett üzenet: olyan információ, melynek tartalma megegyezik a nyílt üzenetével, eltérve attól abban, hogy csak a feladó és a címzett tudja visszaalakítani olvasható nyílt üzenetté. Ebben a formában tehát más szereplő számára nem értelmezhető. Ilyen alakban az üzenet megbízhatatlan csatornákon is továbbítható, legrosszabb esetben is csak sikertelenül (a megmásított, azaz hamisított információ közlése vagy annak kiszivárgása helyett).
- Aláírás: a nyílt üzenethez tartozó kiegészítő jellegű információ, amivel az üzenet sértetlensége megállapítható és a feladójának kiléte igazolható.
- Kulcs: az a bizalmas információ, ami ahhoz szükséges, hogy a nyílt üzenetből rejtjelezett üzenetet és/vagy arra vonatkozó aláírást állítsunk elő - vagy a rejtjelezett üzenetből nyílt üzenetet fejtsünk meg és/vagy ellenőrizzük az arra vonatkozó aláírást.
- Támadó: olyan köztes szereplő a kommunikációban, aki a rá nem tartozó nyílt üzenet, vagy akár a kulcs illetéktelen megszerzésére és/vagy az üzenet útközbeni módosítására, ezáltal a szereplők manipulálására törekszik. Amennyiben a támadó a kulcshoz hozzájut, minden további üzenetet el tud olvasni, és néhány esetben visszamenőleg meg tudja fejteni a korábban begyűjtötteket. Az előbbieken túl képes lesz aláírások hamisítására. Ezeket teljesen észrevétlenül teheti.

A fenti fogalmakkal leírható az úgynevezett szimmetrikus kulcsú és az aszimmetrikus kulcsú titkosítás alapelve. Az általános felírás előtt szerepeljen egy példa, a fogalmak intuitív megértését elősegítendő.

2.1. Példa szimmetrikus kulcsú titkosításra: Caesar-rejtjel

Ahhoz, hogy gyakorlati tekintetben is átláthatóak maradjanak a fenti fogalmak, tekintsük az egyszerű „betűeltolásos”, (szimmetrikus) titkosítást. A pontosságra törekedve:

- Legyen egy tetszőlegesen választott l elemű rendezett jelkészletünk - például az angol ABC karakterei - melynek j-edik eleme legyen $\alpha(j)$.
- Legyen $\alpha^{-1}(\iota)$ függvény, ami α rendezett jelei közül ι-hoz annak sorban elfoglalt helyét rendeli hozzá.
- Legyen a *nyílt üzenet* egy tetszőleges jelekből álló rendezett lánc, i-edik jele $M(i)$, úgy, hogy $M(i) \in \alpha \forall i$.
- Legyen a *rejtjelezett üzenet* i-edik jele $S(i)$, úgy, hogy $S(i) \in \alpha \forall i$.
- Válasszunk egy $\kappa \in \mathbb{N}$ számot, ez legyen *a titkosítás kulcsa.* Érdemes κ-t úgy választani, hogy $1 \leq \kappa < l$ igaz legyen.

Ebben az esetben a nyílt üzenetből a rejtjelezett üzenetet κ birtokában tudjuk előállítani.

$$S(i) = \alpha\left(\alpha^{-1}\left(M(i)\right) + \kappa\right) \tag{1}$$

Figyeljük meg, hogy ezt a módszert κ ismeretében a másik irányba is tudjuk alkalmazni, azaz elő tudjuk állítani a titkosított üzenetből a nyílt üzenetet.

$$M(i) = \alpha\left(\alpha^{-1}\left(S(i)\right) - \kappa\right) \tag{2}$$

Világosan látjuk, hogy S-t miért hívjuk rejtjelezett üzenetnek - ezen formájában értelmezhetetlen, de visszaalakítható a kulcs birtokában nyílt üzenetté.

2.1. állítás: S az M üzenet aláírása is egyben
A 2. pontban aláírást úgy határoltuk be, mint azt az információt, ami az üzenet sértetlenségéről tanúskodik, illetve a feladó kilétét igazolja. Amennyiben adott, hogy κ valóban ismeretlen a támadó számára - és a fenti példát tekintve kis túlzással feltételezzük, hogy nagyon nehéz megfejtenie ezt a számot - akkor a szereplőkről egyértelműen kiderül, hogy kizárólag a feladó és a címzett tud ilyen κ paraméterű értelmezhetetlen üzenetet előállítani.
Ez azt jelenti, hogyha κ-t csak két szereplő ismeri, és a címzett üzenetet kap valahonnan, amit κ-t használva sikeresen meg tud fejteni, akkor egyértelműen tudhatja, hogy ki küldte azt - nem más, mint az egyetlen lehetséges feladó, aki ismeri κ-t. Tehát jelen esetben S a rejtjelezett üzenet és annak aláírása is egyben.

2.2. Általánosabb titkosítási terminológia

Az alábbi elemekkel bővítjük a 2.1. pontban ismertetett jelöléstant:

Titkosító függvény:

$$h : M \mapsto H \tag{3}$$

Visszafejtő függvény:

$$h^{-1} : H \mapsto M \tag{4}$$

Aláíró függvény:

$$s : M \mapsto S \tag{5}$$

Aláírás ellenőrző függvény:

$$s^{-1} : S \mapsto 0; 1 \tag{6}$$

Ahol:

- M - Nyílt üzenetek halmaza,
- H - Rejtjelezett üzenetek halmaza,
- S - Aláírások halmaza,
- κ - Titkosító függvény paramétere (kulcs),
- ξ - Aláírás ellenőrző függvény paramétere (kulcs),
- λ - Visszafejtő függvény paramétere (kulcs),
- σ - Aláíró függvény paramétere (kulcs).

2.2.1. Szimmetrikus kulcsú titkosítás általánosan

Miután azonosítottuk a fenti fogalmakat a Caesar-rejtjelben, mint szimmetrikus kulcsú titkosítás esetén, megérthetjük, miért szimmetrikus a kulcs (2.1. definíció):

2.1. definíció: Ebben a dolgozatban azt mondjuk, hogy egy titkosítási algoritmus szimmetrikus, amennyiben leírható a 2.2. pontban ismertetett általánosabb titkosítási terminológiával, és fennáll rá, hogy:

$$\kappa = \chi = \sigma = \xi \tag{7}$$

Ennek a rendszernek előnye az egyszerűségében, míg hátránya a kulcsok terjesztésének lehetőségeiben rejlik.

Ha szereplők egy csoportjával egy ilyen algoritmust felhasználva szeretnénk kiépíteni a biztonságos kommunikációs közeget, akkor a kulcsok egyeztetésének a kérdését alapvetően két irányból közelíthetjük meg:

- Az egyik megoldásbeli véglet az lehet, hogy a szereplők páronként kulcsot egyeztetnek, azaz, ahány él van a csoport tagjaiból képzett teljes gráfon, annyi különböző kulcsot hoznak létre és cserélnek páronként. Hátránya ennek, hogy sok kulcs cserélés szükséges; a csoport minden tagjának találkoznia kell páronként személyesen minden taggal.
- A csoport megbeszélhet közös kulcsot is, vagy a csoporton belül kisebb csoportok osztozhatnak egy-egy közös kulcson. Ennek sajnos szintén megvannak a maga hátrányai:
  - Ha csak egyetlen tagtól is kiszivárog a közös kulcs, akkor az érintett csoport biztonságos kommunikációra való törekvése meghiúsul - akár visszamenőleg is.
  - Bár már nem szükséges mindenkinek mindenkivel személyesen találkoznia a kulcs egyeztetéséhez, még így is el kell juttatni a közös kulcsokat az érintett csoportoknak.
  - Az üzenet megfejthetősége már nem igazolja a konkrét személyt, csak a küldő csoportbeli tagságát.

Az ilyen jellegű problémákra ígéretes új megoldási lehetőséget nyújtott az aszimmetrikus titkosítás kidolgozása, és annak első megvalósításai. [24]

2.2.2. Aszimmetrikus titkosítás általánosan

Ismerünk olyan titkosítási és hitelesítési algoritmust, amiben a (7) képletben foglaltak közül csak ez a két állítás igaz:

2.2. definíció: Ebben a dolgozatban azt mondjuk, hogy egy titkosítási algoritmus aszimmetrikus, amennyiben leírható a 2.2. pontban ismertetett általánosabb titkosítási terminológiával, és fennáll rá, hogy:

$$\kappa = \xi \tag{8}$$

$$\chi = \sigma \tag{9}$$

$$\kappa \neq \chi \tag{10}$$

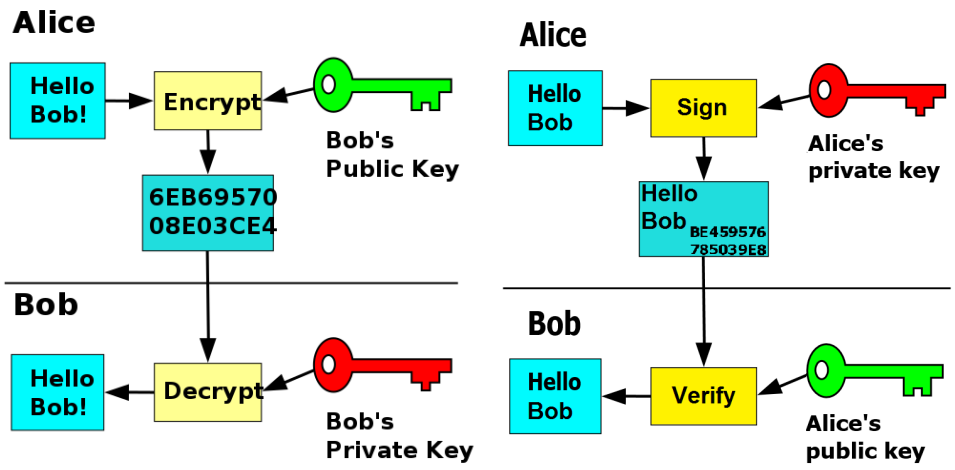
Ez tehát azt jelenti, hogy nem ugyanazzal a kulccsal titkosítunk, mint amivel visszafejtünk, illetve, nem ugyanazzal a kulccsal ellenőrizzük az üzenetre vonatkozó aláírást, mint amivel elkészítjük azt. Ugyanakkor megegyezik az a kulcs, amivel az üzenetet megfejtjük és aláírunk, illetve az a kulcs, amivel titkosítunk és ellenőrzünk egy aláírást.

Az úgynevezett aszimmetrikus vagy nyílt kulcsú titkosításnak az elsők között lévő és egyik legismertebb megvalósítása a modulus aritmetikán alapuló RSA algoritmus [24].

A nyílt kulcsú titkosítás esetén nem kulcsokkal, hanem kulcspárokkal dolgozunk. A kulcspár egy időben generálható, de egyik tagjából a másik tagja nagyon nehezen fejthető meg. Egy-egy ilyen kulcspárt a szereplőkhöz társíthatunk, és az egyik felét ($\kappa = \xi$, a továbbiakban csak κ vagy nyilvános kulcs) kiadhatjuk a nyilvánosságnak. A másik fele ($\chi = \sigma$, a továbbiakban χ vagy titkos kulcs) a létrehozója védelmében marad.

Ha egy csoport minden tagja rendelkezik ilyen kulcspárral, akkor az üzenetváltás és a kulcs egyeztetés leegyszerűsödik. Tekintsük a 2.1. ábrát. Ha Alice Bob-nak üzenetet szeretne küldeni, csak Bob nyilvános kulcsára és a saját titkos kulcsára lesz szüksége. Miután az üzenetet megfogalmazta, a saját titkos kulcsát használhatja arra, hogy előállítson egy aláírást. Bob nyilvános kulcsának segítségével az így keletkezett adathalmazt titkosíthatja, majd elküldheti. Bob megfejtheti az üzenetet a saját titkos kulcsával, és ha ismeri Alice nyilvános kulcsát, meggyőződhet róla, hogy az üzenet valóban Alice-től jött[2].

[2]Feltéve, hogy biztos benne, hogy Alice nyilvános kulcsa, ami nála van, tényleg Alice-hez tartozik, és nem hamisítvány - ez azonban egy külön problémakör.



2.1. ábra. Nyílt kulcsú titkosítás és aláírás vázlatosan

(licenc: közkincs[3])

Láthatjuk, hogy ebben a rendszerben alapvetően kétféle érzékeny információval dolgozunk - ezek:

- A védendő üzenet
- A (titkos) kulcs

A továbbiakban a fentiek közül a kulcs védelmével foglalkozunk - az idevezető okfejtést a 3. pontban ismertetjük.

[3] `https://en.wikipedia.org/wiki/Public-key_cryptography`, elérés dátuma: 2020. 11. 08.

3. Hitelesebb digitális aláírásokért

A titkosított és hitelesített kommunikáció biztosításakor figyelembe vesszük, hogy a kulcson túlmenően maga az üzenet is védendő adat. Mindazonáltal, a kulcs bizalmasságát az üzenettel szemben kritikusabbnak ítéljük meg, hiszen ennek illetéktelen birtokában a titkosítás akár visszamenőleg is megfejthető (általában észrevétlenül), míg ezzel egy időben a hitelesítést szolgáló aláírások hamisíthatóvá válnak.

Azokban az esetekben, amikor a kommunikációban az információ hitelességét (sértetlenségét) tekintjük kritikusnak, és a bizalmasságot elhanyagolhatónak (például nagyobb közönségnek címzett, vagy nyilvánosan elérhetőnek szánt információ közreadásakor), az üzeneteket elegendő digitális aláírással ellátni. Utóbbiak generálásakor a megoldandó probléma egyszerűbbé válik olyan tekintetben, hogy a 2.2.2. pontban említett érzékeny információk közül elégséges a kulcs védelmét biztosítani.

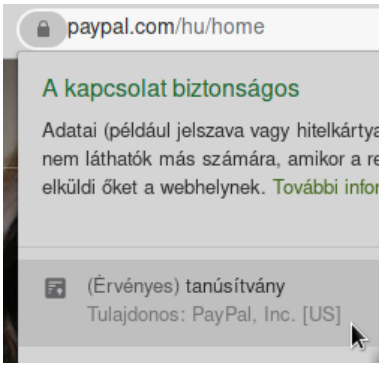
A dolgozat innentől kezdve elsősorban az üzenetek hitelesítésekor, másodsorban a titkosított üzenetek visszafejtése során használt kulcs védelmével foglalkozik.

3.1. Hol alkalmazunk digitális aláírásokat?

Szerepeljen itt néhány példa a teljesség igénye nélkül a napjainkban elterjedt alkalmazásokról. Digitális aláírásokat alkalmazunk többek között:

- Internetes portálok hitelesítésére az ott olvasható adatok eredetiségének szavatolására, biztonságos letöltésekhez, vírusfertőzések megelőzéséhez.
  - Internetes portálok esetében a webhelyen felül a vállalat azonosításához (3.1. ábra) (lásd: EV SSL).
- Szoftver telepítőcsomagok aláírására - szintén a vírusfertőzés megelőzőséhez, és/vagy a szerző azonosításához.
  - Windows operációs rendszeren futtatható fájlok (.exe) és telepítőcsomagok (.msi) aláírására.
  - GNU/Linux operációs rendszereken .deb, .rpm, .pkg.tar.xz, stb. fájlok vagy csomagtárolók digitális aláírására.
- Vállalati magánhálózatba, adminisztrációs felületekre való fokozott biztonságú belépéshez.

- Hivatalos dokumentumok állam előtt elismert digitális aláírásához (lásd: GOV CA).
- Magáncélra - az üzenet hitelesítésére.
  - Elektronikus levelek (e-mail).
  - Csevegő platformok, webhelyek esetén vagy a központtal való kapcsolattartáskor, vagy végponttól végponti kapcsolat esetén a partnerek üzeneteinek (automatikus, háttérben történő) hitelesítésére (pl. XMPP / OTR protokoll).
  - Biztonságos VoIP kapcsolat felépítésére (SIP / SRTP, ZRTP).
- Egyéni esetekben, bármilyen fájl hitelesítésére.



3.1. ábra. Vállalatot azonosító EV SSL tanúsítvány

3.2. GnuPG kulcsok védelme

A 3.1. pontban felsoroltak közül mi a közösségi alapú GnuPG titkosító programmal és annak egy speciális alkalmazásával, a GNU/Linux rendszerekhez összeállított szoftvercsomagok hitelesítésével foglalkozunk.

3.2.1. GnuPG és OpenSSL

Azért helyezünk nagyobb figyelmet a GnuPG programra a Public Key Infrastructure (PKI) alapú titkosító programokkal szemben - mint amilyen például az OpenSSL - mert tapasztalatunk szerint könnyebben kezelhető, és nem a szerveroldali rendszergazdát, hanem a mindennapi felhasználót feltételezi alkalmazójaként.

További szimpátiát kelt bennünk, hogy a Web of Trust (WoT) modellben kulcsok hitelesítésében a közösség bármely tagja könnyedén szerepet vállalhat, ellentétben a PKI modellel, ahol a mindennapi felhasználót tudását nagyban meghaladó szakértelem szükséges az előírt infrastruktúra kiépítéséhez (többszintű CA, CRL, stb.), vagy az előbbi helyett harmadik fél bevonása, ahol már készen áll a PKI rendszer.

3.2.2. A csomagokról és a csomagkezelőről röviden

A legtöbb Linux disztribúció a rendszer komponenseinek telepítéséhez és eltávolításához úgynevezett csomagkezelő programot használ. A csomagkezelő egy távoli szerverről - egy úgynevezett tárolóból - keresi és telepíti a kért szoftvereket, illetve azok függőségeit. A csomagkezelő a telepített csomagokat nyilvántartja és lehetővé teszi azok automatikus frissítését vagy eltávolítását.

Ezeket a csomagokat a biztonság fokozásáért, fejlesztőik digitális aláírással látják el. A rendszer frissen telepített állapotában már tartalmazza azoknak az aláíró kulcsoknak a nyilvános párjait, amivel a későbbi csomagokat - vagy a meglévők frissítéseit - ellenőrizni fogja. Ezzel képes megelőzni a hálózaton meghamisított csomagok telepítését, ami adott esetben rosszindulatú kódot tartalmazhat.

A legelterjedtebb csomagformátumok közül a .deb és a .rpm csomagokat a GnuPG titkosító programmal lehet aláírni. Ennek a munkafolyamatnak az alapsérülékenysége már ismert: a kulcsot a támadható végpont tárolja - lásd az 1.3.1. pontot. Ha ez kiszivárog, akkor már nem lehet tudni a későbbiekben, hogy melyik aláírásokban lehet megbízni.

3.3. A végpont elválasztása a kritikus felelősségtől - lehetőségek

Az 1.4. pontban azt a megoldási lehetőséget fontoltuk meg, hogy ahelyett, hogy a végponton tárolnánk az érzékeny információt - jelen esetben a kulcsot -, inkább külön erre a célra kijelölt hardvert használunk fel.

Ez valójában nem saját gondolat, hiszen a 3.3.3 pontban ismertetett ún. intelligens kártyák (lásd: intelligens kártya) ezt a működést valósítják meg.

A problémát többféleképpen megközelíthetjük:

1. Kijelölhetünk egy dedikált számítógépet (lásd: 3.3.1. pont).
2. Használhatjuk ugyanazt a számítógépet rendkívül körültekintően (lásd: 3.3.2. pont).

3. Használhatunk egy OpenPGP intelligens kártyát (lásd: 3.3.3. pont).
    - Felprogramozhatunk egy régi eszközt, ami ellátja az intelligens kártyák funkcióit (lásd: 3.3.4. pont).

3.3.1. Megoldás erre a célra kijelölt számítógéppel

Szükségünk van ehhez:

- Egy nem használt számítógépre, amit kijelölhetünk *erre a kizárólagos célra.* Szóba jöhet akár egy dedikált Raspberry Pi vagy hasonló kis fogyasztású ARM processzoros alaplap.
- Biztonságos felhasználói kommunikációs felületre a másik gép felé - lehetőségek:
    - SSH vagy egyéb interaktív kapcsolat természetesen nem jön szóba, hiszen ebben az esetben a mi hálózatra kötött végpontunk rosszindulatú viselkedésének abban a pillanatban ki lett téve a kulcstároló számítógép is.
    - Másik monitor és billentyűzet.
    - Hely az elsődleges gép mellett és az alábbiak egyike:
        * Úgynevezett KVM kapcsoló.
        * Sok türelem a kábelek rendszeres átrendezésére.
- Valamilyen hordozható háttértárra az aláírandó, illetve aláírt adat hordozására.

A legegyszerűbb az opciók közül, ha a kijelölt számítógépre feltelepítünk egy azt támogató Linux disztribúciót amit nem fogunk kommunikációs hálózatra csatlakoztatni. Érdemes ezért eltávolítani, de legalább letiltani a hálózati kártya kezelését a rendszerben. A merevlemez (vagy mini alaplap esetén gyakran SD kártya) tartalmát célszerű eleve titkosítottan őrizni, amit rendszerbetöltéskor a betöltő program felold.

A háttértárra felmásolhatjuk a csomagokat vagy más egyéb aláírandó adatokat, aláírhatjuk azokat, majd visszavihetjük az aláírásokat az online végpontra. A végpont sérülékenysége esetén így a kulcs kompromittálódását nem kockáztattuk.

Fő előnyei a megközelítésnek, hogy:

- elegendő védelmet biztosít, hiszen az online végpont sérülése esetén a kulcs még mindig biztonságban van,

- néhány esetben az eszközök rendelkezésre állnak: régi, már nem használt számítógépet, amin mindösszesen a grafikus asztal nélküli GNU/Linux disztribúciót szeretnénk futtatni, jó eséllyel találhatunk.

Hátránya pedig, hogy aránylag sok munkát igényel a beüzemelés és a rutinszerű használat, kényelmetlen. Az írás pillanatában nem tudtunk olyan Linux disztribúcióról, ami ezt a megoldást készen és automatizáltan kínálja.

A probléma egyik lehetséges orvoslására a GnuPG támogatja az úgynevezett alkulcsok használatát. A Debian Linux közösségi dokumentációja az alábbi szövegrészlettel mutatja be ugyanezt a problémát:

> However, keeping all your keys extremely safe is inconvenient: every time you need to sign a new package upload, you need to copy the packages onto suitable portable media, go into your sub-basement, prove to the armed guards that you're you by using several methods of biometric and other identification, go through a deadly maze, feed the guard dogs the right kind of meat, and then finally open the safe, get out the signing laptop, and sign the packages. Then do the reverse to get back up to your Internet connection for uploading the packages. [26]

3.3.2. Megoldás ugyanazzal a számítógéppel

Ehhez szükségünk van:

- a meglévő végpontra,
- egy - ideálisabb esetben hordozható - háttértárra, amiről indíthatunk rendszert.

A háttértárra feltelepíthetünk egy titkosított hordozható Linux disztribúciót, amin a védendő kulcsokat őrizzük. Amikor adatot szeretnénk aláírni, akkor a számítógépet kikapcsoljuk, leválasztjuk a hálózati kábelt vagy vezeték nélküli elérés esetén letiltjuk a hálózati perifériát a BIOS beállításokban.

Ezután elindíthatjuk a rendszert a háttértárról. Ez betöltéskor el fogja tőlünk kérni azt a jelszót, amivel a rajta lévő adatokat titkosítottuk[4]. A futó rendszeren felcsatolhatjuk a helyi számítógép merevlemezén elérhető adatokat és elvégezhetjük az aláírásokat. A művelet

[4]Ez egyébként az eddig ismertetett terminológia szerint egy szimmetrikus kulcs.

után a számítógépet kikapcsoljuk, a háttértárat eltávolítjuk, végül a hálózati perifériát ismét engedélyezhetjük és használhatjuk a rendszert.

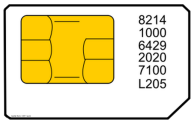
Előnye ennek a megoldásnak, hogy akár egy SD kártya vagy egy pendrive is elegendő a célra. Hátrányai, hogy:

- Rendkívül kényelmetlen üzembe helyezni.
- Rendkívül kényelmetlen és időigényes használni.
- Könnyű hibázni:
  - El lehet felejteni letiltani a hálózatot.
  - A titkosítást figyelmetlenségből vagy meggondolatlanságból fel lehet oldani a végpont saját operációs rendszerén is (sok disztribúció grafikus felülete beépítetten támogatja) - ekkor a végpontnak kiadtuk a jelszót. Ebben a pillanatban ugyanott tartunk, mintha eleve végig a végponton tároltuk volna a kulcsainkat.
  - Szintén figyelmetlenségből vagy meggondolatlanságból könnyű érzékeny adatot felmásolni a végpont alapértelmezett rendszerére.
  - Ha a háttértárat bent felejtjük rendes rendszerindítás során, akkor a végpont megfertőzheti az azon lévő rendszerbetöltőt, ami a következő indításkor kimentheti a háttértár betöltőjénél átadott titkosító jelszót. Amikor ismét a végpont saját operációs rendszerét indítjuk, akkor az kiolvashatja ezt és megkísérelheti feloldani.
    * Mivel az ismertetett technikát kézzel állítottuk be, ezért kicsi az esélye annak, hogy létezik ennek egy féreg formájában nagyközönséget támadó automatizált változata. Annak az esélye, hogy célzott támadás ér minket, meglehetősen alacsony, ezért egyéni célra ez a biztonsági szint már valószínűleg megfelel. Hasonlóan részletesen előre megtervezett támadás mindenesetre lehetséges. [28] [2]

3.3.3. Megoldás intelligens kártyával

Léteznek olyan célhardverek, úgynevezett intelligens kártyák, amiknek egyetlen feladata ezt a problémát megoldani. Egy bizonyos fajtájukat a telefonunkban is megtalálni.

Az intelligens kártya lényegében egy integrált áramkör, ami hasonlóan viselkedik egy rendkívül kis tárolókapacitású háttértárhoz - azzal ellentétben azonban képes bizonyos számítások elvégzésére, és döntéseket hozni afelől, hogy milyen információkat ad ki.



3.2. ábra. Intelligens kártya

(licenc: közkincs[5])

Tervezésekor azt is figyelembe veszik, hogy műveleteinek végrehajtása során a hardver fizikai megvalósításából adódóan milyen információkat szivárogtat a környezetébe. Ideális esetben például a kártya teljesítményfelvételéből nem lehet következtetni az azon tárolt adatokra, az elvégzett műveletre, vagy annak paramétereire. [30]

Használat előtt a hardvernek beazonosíthatjuk magunkat - tipikusan egy PIN kód megadásával - ami után az lehetőséget adhat azon műveletek végrehajtására, amik csak a titkos kulcs birtokában lehetségesek. Végül az eszköz a művelet eredményét elküldi - közvetlen csatornán keresztül - a nehezebben védhető végpontnak. Magát a kulcsot azonban nem adja ki - ilyen instrukciót ugyanis nem támogat.

Ennek az eszköznek az előbbiekhez képest nagyon kényelmes a használata. A végponton telepített szoftverek anélkül képesek vele kommunikálni, hogy azok ismernék a kártya megvalósításának részleteit - fizikai szinten, annak szoftverét, vagy a hozzá tartozó olvasó működési elvét. Ezt egy előre kidolgozott protokoll teszi lehetővé, ami meghatározza a kártya viselkedésével szembeni elvárásokat az azt kezelő alkalmazás irányából.

A GnuPG szoftver támogatja az intelligens kártyák kezelését egy saját protokollon keresztül, ezt specifikálja az úgynevezett OpenPGP Smart Card szabvány. [22]

Előnye az utóbbi szabványt megvalósító intelligens kártyának is, hogy kényelmesen és biztonságosan használható. Hátrányaihoz tartozik, hogy:

- Ezek a kártyák sajnos kevésbé elterjedtek, mint a PKI tanúsítvány alapon működő hardveres kulcsok.
- A szabvány csak három tárolható kulcsot definiál egy ilyen kártyán: egyet titkosításra, egyet hitelesítésre, egyet aláírásra - ellentétben például a PKI modellt követő PKCS#11 szabványt megvalósító intelligens kártya illesztőkkel. [23]
- A szabvány csak a kulcs algoritmus-specifikus paramétereinek tárolására biztosít lehetőséget. A fejlécadatok nélkül a GnuPG kulcs önmagában nem használható.

[5] `https://publicdomainvectors.org/photos/sim-card-finish.png`, elérés dátuma: 2020. 11. 08.

- Az írás pillanatában a szabvánnyal kompatibilis kártyák ára általában magas volt ($\approx$ 18 EUR[6]), és ezért a csomag aláíró fejlesztők és más GnuPG felhasználók többségének nem valószínű, hogy szóba jöhető alternatíva.

Az intelligens kártya aláírásra, és titkosítás feloldására is alkalmazható

A visszafejtés erőforrás igénye összemérhető az aláírás elkészítésének erőforrás igényével.

Ahogyan a kártyának nem kell az egész adathalmazt aláírnia, csak egy arról készült ujjlenyomatot, ehhez hasonlóan annak nem kell a teljes adathalmazt dekódolnia sem. Amikor a GnuPG segítségével titkosítunk egy fájlt egy nyilvános kulcsra, akkor valójában egy helyben generált, egyszerű szimmetrikus kulcsra rejtjelezünk először. A GnuPG ezután az imént használt kulcsot titkosítja a címzett nyilvános kulcsára, majd mellékeli azt a „hasznos" adathalmazhoz.

Ennek a technikának előnye, hogy több címzett nyilvános kulcsot is megadhatunk a fájl növekedése nélkül, illetve, a kártyának csak rövid terjedelmű - néhány kilobájt nagyságrendű - adatot kell visszafejtenie. [7]

3.3.4. Megoldás erre kijelölt hardverrel és intelligens kártya emulációval

Az intelligens kártya a 3.3.3. pontban ismertetett hátrányainak ellenére közel megfelelő megoldásnak bizonyul az általunk keresett biztonság ergonomikus megvalósításához (a TDK munka keretében a fizikai mérésekből kiinduló támadásokkal nem foglalkozunk).

Ezek közül sok problémát az erőforrásokban jóval gazdagabb számítógép - mint védett kulcstároló eszköz - nem hordoz magában. A 3.3.1. pontban láthattuk, hogy a kijelölt számítógép funkcióinak megvalósításához nincs szükség nagy számítási kapacitással bíró hardverre.

Ez többek között abból is adódik, hogy digitális aláírás készítésekor a kártya egy rögzített hosszúságú dokumentumlenyomatból állít elő egy szintén rögzített hosszúságú igazoló lenyomatot - azaz az aláírandó állomány mérete nem tekinthető befolyásoló tényezőnek.

Felmerült bennünk a kérdés, hogy tulajdonképpen van -e akadálya annak, hogy egyszerűbb programozható hardvereinket dedikált kulcstároló eszközként alkalmazzuk? Ha egy ilyen eszközt emuláció útján az intelligens kártyához hasonló módon tudnánk használni, akkor mindkét megoldásnak - a 3.3.1. pontban tárgyalt dedikált számítógéppel megvalósított és a 3.3.3. pontban ismertetett intelligens kártyás megoldásnak is - számos előnyét megörökölhetnénk.

[6] pl. `https://www.floss-shop.de/en/security-privacy/smartcards/13/openpgp-smart-card-v3.3`

Ahhoz, hogy ezt a kérdést megválaszoljuk, pontosan tudnunk kell, hogy mit várunk egy intelligens kártyától.

Elvárásaink az intelligens kártyával szemben

- A kártyának legfeljebb kettő kommunikációs interfésze legyen: egy a számítógép felé, egy a felhasználó irányába (utóbbi: pinpad, ujjlenyomat olvasó).
  - A kapcsolatokkal szemben elvárás az is, hogy amennyire lehetséges, közvetlen legyen, ezért az eszköznek nem szabad megosztottan használt kommunikációs hálózatra kapcsolódnia.
- A kártya, amennyiben megoldható, ne szivárogtasson a környezetébe olyan a működésére utaló mérhető jeleket, amikből annak belső állapotára lehet következtetni.
- A kártya legyen képes titkos kulcsok tárolására, azok segítségével aláírásokat előállítani és titkosítást visszafejteni.
- *A kártyának a kliens oldalon - a számítógépen - legyen szoftveres támogatottsága, így elérve, hogy a használata automatikus és kényelmes legyen.*

A 3.3.1. pontban láthattuk, hogy ezeket az elvárásokat - a fizikai lehallgathatóság elleni védelem kivételével, mely túlmutat a dolgozat szempontjain - egy kizárólagosan erre a célra kijelölt számítógép is meg tudja valósítani.

A felsorolás utolsó pontjában megfogalmazott elvárásra adott (közösség számára elérhető, nyílt forrású) megoldásról az írás pillanatában nem volt tudomásunk. Utóbbi hiányában csak az állományok külső háttértáron történő manuális szállításával használható az elkülönített aláíró/titkosítást visszafejtő számítógép.

Köszönhetően azonban a tömegesen gyártott hardvereknek - aminek gyakorlatával a szerzők nem feltétlenül értenek egyet -, ma már sokunk háztartásában találhatóak olyan eszközök, ami a fentebbi pontokban megfogalmazott elvárásokat teljesítésére akár alkalmasak is lehetnének és pillanatnyilag kihasználatlanok. Ezek elsősorban a régi okostelefonjaink.

Alkalmazható megoldás

Az egyéni kulcstároló eszközök által működtetett virtuális intelligens kártyák előnyeihez sorolhatjuk, hogy:

- létrehozásuk sok esetben kiadás nélkül járhat, hiszen rengeteg korai Android eszköznek adhatnánk új felhasználást,
- a kulcs méretéből adódóan gyakorlatilag tetszőleges mennyiségű kártyának az információit lehetne hordozni egy telefonon.

Az okostelefonnal megvalósított kulcstároló eszköz hátrányaihoz tartoznak, hogy:

- a hardver további hálózati kommunikációs interfészekkel bír (pl. Bluetooth és WiFi periféria),
  - Lehetséges megoldás erre, hogy az ezen perifériák okozta kockázatot manuálisan szüntetjük meg. Ezt elméletileg megtehetjük akár a hardver károsításával vagy leárnyékolásával, esetleg szoftveres úton az illesztőprogramok eltávolításával.
- mivel az ARM processzoros alaplapú telefonokat nem biztonsági modulnak tervezték, ezért fizikai környezetére való hatásain keresztül közvetlen mérések útján esetleg következtetni lehet annak belső állapotára.
  - Ez gyakorlati tekintetben annak veszélyét hordozza magában, hogy az elveszített eszköz adatainak feltárása sokkal egyszerűbben lehetséges, mint ugyanezen információk egy intelligens kártyából történő kiolvasása.
  - Nincsen tudomásunk az írás pillanatában arról, hogy egy kémprogram a végponton elegendő információhoz tud -e hozzájutni a külső eszközről - például annak teljesítményfelvételéről az idő függvényében -, hogy abból érzékeny információkra tudjon következtetni.

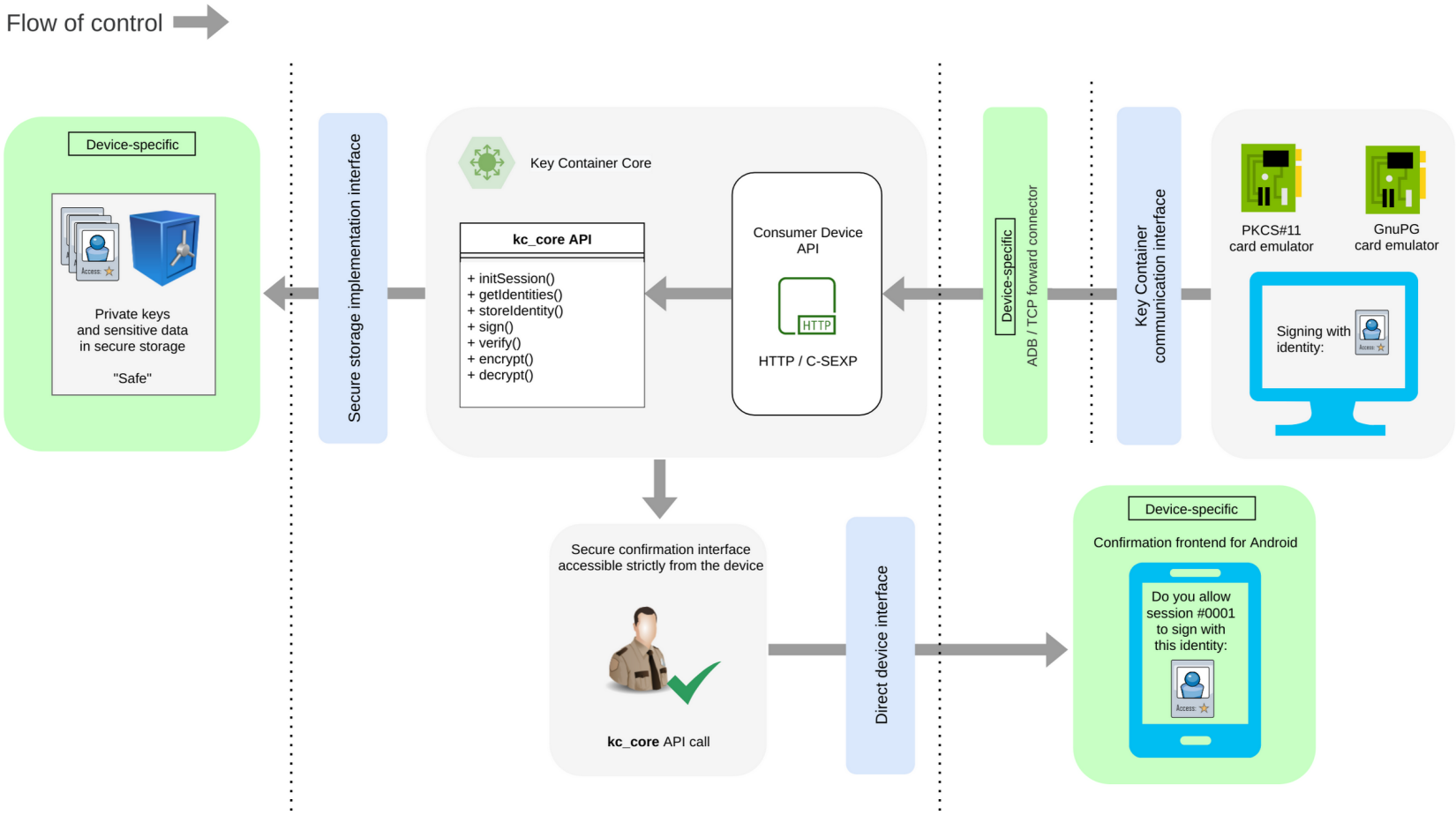
Úgy gondoljuk, a fentiek ellenére biztonsági tekintetben sokkal jobb körülményeket teremthetünk a kiforrott megoldás alkalmazásával, mint azáltal, hogy a kulcsot közvetlenül a végponton tároljuk. Végső soron azt szeretnénk elérni, hogy a megoldásunk hozzáférhetőbb alternatíva legyen a GnuPG felhasználói számára, mint az intelligens kártya beszerzése.

4. A kulcstároló architektúra

Miután megfogalmaztuk, hogy milyen problémára szeretnénk alternatív megoldást adni az érdeklődők számára, akár egy okostelefon vagy egyéb egyszerű, kulcstároló eszköznek kinevezett hardver segítségével, összefoglaljuk a megvalósításra vonatkozó alapvető elgondolásunkat.

A felhasználás módjának egy egyszerűsített áttekintése a 4.2. ábrán látható. Azon kivitelezési szempontok, amik a projekt jelenlegi szakaszában tisztázódni látszódnak, a következők:

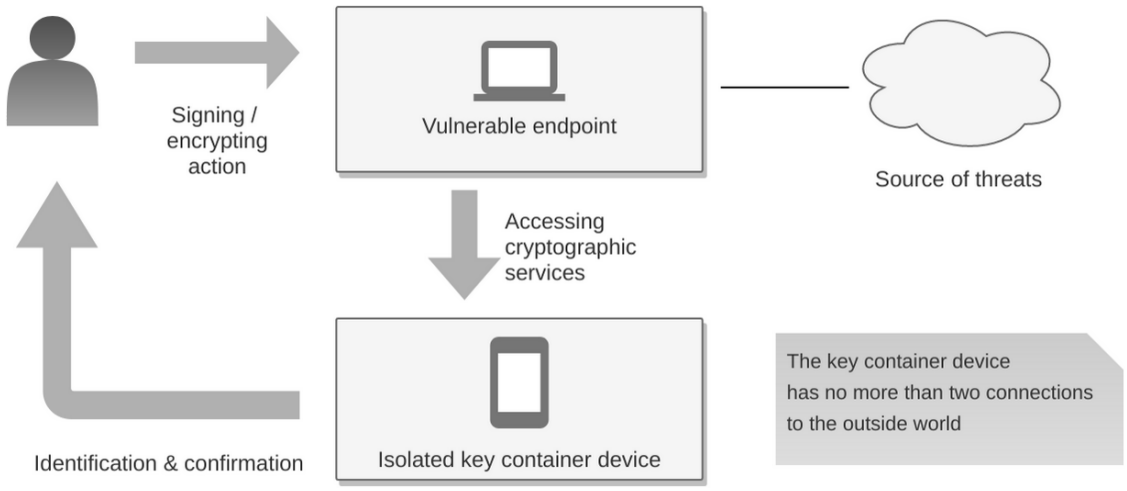
- Ahogyan azt a 3.3.4. pontban lefektettük, a kulcstároló működést megvalósító eszköztől (4.1. ábra, „Key Container Core") - akárcsak egy intelligens kártyától - azt várjuk, hogy legfeljebb kettő kommunikációs interfésze legyen. Az első felületén keresztül tartsa a kapcsolatot az emulációt megvalósító végponttal, míg a második, (opcionális) felhasználói perifériájának segítségével kérjen megerősítést, PIN kódot, vagy egyéb visszajelzést a lehető legközvetlenebb úton (4.1. ábra: „Direct device interface").
- Azért, hogy a későbbi vagy más kategóriába eső eszközökre a projekt minél nagyobb eséllyel legyen átültethető, annak hordozható (platformfüggetlen) részeit különválasztjuk, és C nyelven írjuk.
  - A kulcstároló működést megvalósító, hordozható mag (4.1. ábra: „Key Container Core", és a szaggatott vonalon belüli középső rész) egy választott eszközre történő átültetéséhez három illesztés implementációja biztosan szükséges lesz: az interfésznek a végpont irányába (4.1. ábra: „Key Container communication interface"), az interfésznek a felhasználó irányába a közvetlen felületen keresztül (4.1. ábra: „Direct device interface"), és a biztonságos adattárolásnak a megvalósítása (4.1. ábra: „Secure storage implementation interface") - kihasználva így például az Android ökoszisztéma alkalmazás-specifikus tárhely biztosítására igénybe vehető fejlesztői API felületet). További példák a 4.1. táblázatban olvashatóak.
- A projekt általában a kis teljesítményű, Linux alapú operációs rendszereket futtatni képes eszközöket igyekszik hasznosítani - a kulcstároló eszköz ennélfogva lehetne akár egyéb kis teljesítményű processzorral ellátott alaplap, másik számítógép, vagy egyéb programozható készülék is.
  - Az eszköz típusát egyfelől azzal indokoljuk, hogy arra számítunk, hogy használaton kívüli okostelefonokból vagy egyéb okos eszközökből a háztartásokban az idő előrehaladtával egyre több lesz; másfelől az operációs rendszer jelenléte lehetőséget teremt már létező, összetettebb, C nyelven íródott könyvtárak használatát.



4.1. ábra. A kulcstároló eszköz kapcsolatai a külvilággal és a gazda hardverrel - Android eszköz esetén

Interfész	Eszköz	Lehetséges implementáció
Végponti kapcsolat	Android készülék	ADB kapcsolat
	Raspberry Pi vagy egyéb mini alaplap	Hálózati interfész, soros port
	Dedikált számítógép	Hálózati interfész
Közvetlen felhasználói kapcsolat	Android készülék	Android alkalmazás / telefon képernyő
	Raspberry Pi vagy egyéb mini alaplap	Nyomógomb, LED-ek
	Dedikált számítógép	Felhasználói program
Biztonságos tár	Android készülék	Android alkalmazás privát perzisztens tára
	Raspberry Pi vagy egyéb mini alaplap	Rendszergazdaként hozzáférhető fájlrendszeri terület
	Dedikált számítógép	Rendszergazdaként hozzáférhető fájlrendszeri terület

4.1. táblázat. Példák a kulcstároló eszköz (`kc_core`) megvalósítására többféle eszközön



4.2. ábra. A kulcstároló eszköz gyakorlati tekintetben éppen egy intelligens kártya szerepét tölti be

A dolgozat hátralévő részében a végponti kártya emuláció megvalósításának kezdeti szakaszainak, és a (leendő) kulcstároló program Android környezetbe való illesztésének ismertetésével folytatjuk.

5. Végponti intelligens kártya emuláció megvalósítása

5.1. Nélkülözhetetlen -e a kártya emuláció?

Mielőtt nagyobb energiát fektetnénk bele, érdemes végiggondolni, hogy valójában a biztonságos kulcshasználathoz nem feltétlenül szükséges megvalósítani egyáltalán az emulációt. Elegendő lehet az a megoldás is, hogy az aláíró vagy titkosítást visszafejtő hardverre az információcserét akadálymentesítjük, automatizáljuk.

Az emuláció létrehozásával azonban elérhetjük, hogy a keretrendszerünk - jelen esetben a GnuPG - a megoldásunkat kompatibilis intelligens kártyaként prezentálja a felhasználói programok irányába. Azt várjuk, hogy ennek következményeként azok a programok, amik közvetetten - a keretrendszeren keresztül - kezelni tudtak intelligens kártyákat, hasonló módon az erre a célra alkalmazott hardveren, azok a segédeszközön futó kulcstároló kezelésére is képesek lesznek. Ezt a lehetőséget egyrészt talán egyszerűbb kiaknázni, másrészt jobb ergonómiai élményt biztosít a felhasználó számára, lévén a már ismert felületeket használhatja a virtuális intelligens kártyához is.

5.2. Az illesztő rétegek áttekintése

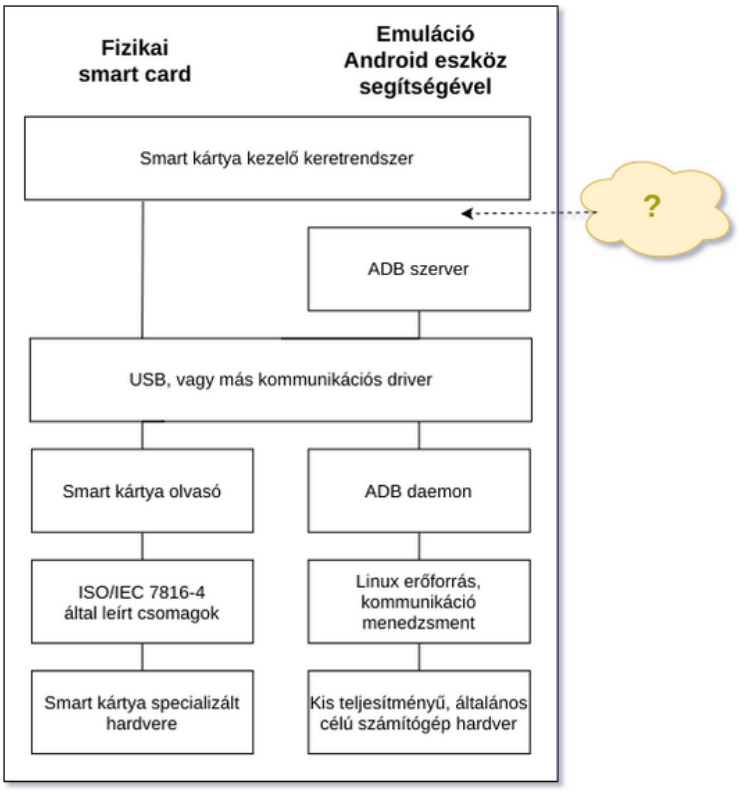
Az intelligens kártyák típusai közötti legnagyobb eltérés a funkciók számában, a támogatott kommunikációs protokollokban, és általában a kártya felépítésben rejlik. [4]. Ahhoz, hogy a különböző kártyákat, különböző olvasókon keresztül, ismételten különböző hardverfelépítésű számítógépeken egységesen tudjuk kezelni, több illesztést is el kell végezni a rendszerben.

Alulról felfelé haladva az 5.1. ábra fizikai intelligens kártyák illesztéseit ismertető oszlopában, a kártya és az olvasó közötti adatcsere rendezett bitsorozatokban, úgynevezett Application Protocol Data Unit (APDU) csomagpárokban történik. Az olvasó egy parancs csomagot küld a kártyának, amire a kártya egy választ ad. A protokollt részleteiben az ISO/IEC 7816-4 szabvány definiálja. [12]

A kártyaolvasó egy illesztőeszköz, ami lehetővé teszi a kártya és számítógép közötti kommunikációt. Az olvasó képes lehet PIN kód, vagy más adatok fogadására is.

A kártyaolvasó és a számítógép közötti adatátvitel egy illesztőprogram segítségével játszódik le. A kártyaolvasó azonban még nem tolmácsolja a számítógépnek az intelligens kártya szoftverének szándékait APDU csomagok formájában[7], csupán a hardveres összeköttetést

[7] Legalább is a GnuPG esetében a forráskódból erre következtettünk.



5.1. ábra. A tényleges és az okostelefon segítségével emulált intelligens kártya kommunikációs illesztései

teremti meg. Ez utóbbi feladatot egy további végpontra telepített illesztőprogram végzi el. Utóbbi PKCS#11 szabvány esetén megosztott függvénykönyvtárak formájában települ a végponti rendszerre, míg az általunk vizsgált GnuPG programnak beépített része [23] - ami azt is jelenti egyúttal, hogy a GnuPG az eredeti formájában kizárólag az általa elvárt APDU protokollt megvalósító kártyákat támogatja.

Az illesztőréteget a rendszerben felhasználói programok vehetik igénybe - böngészők, levelezőprogramok, csomagaláíró szoftverek, stb. Például a PKCS#11 szabvány esetében a böngésző a megfelelő .so vagy .dll állomány rutinjait használja fel, míg Linux csomagok aláírásakor a GnuPG közvetlen működtetésével, többnyire a `libgpgme` („GnuPG Made Easy") API könyvtáron keresztül érheti el az alkalmazás a kártyát.

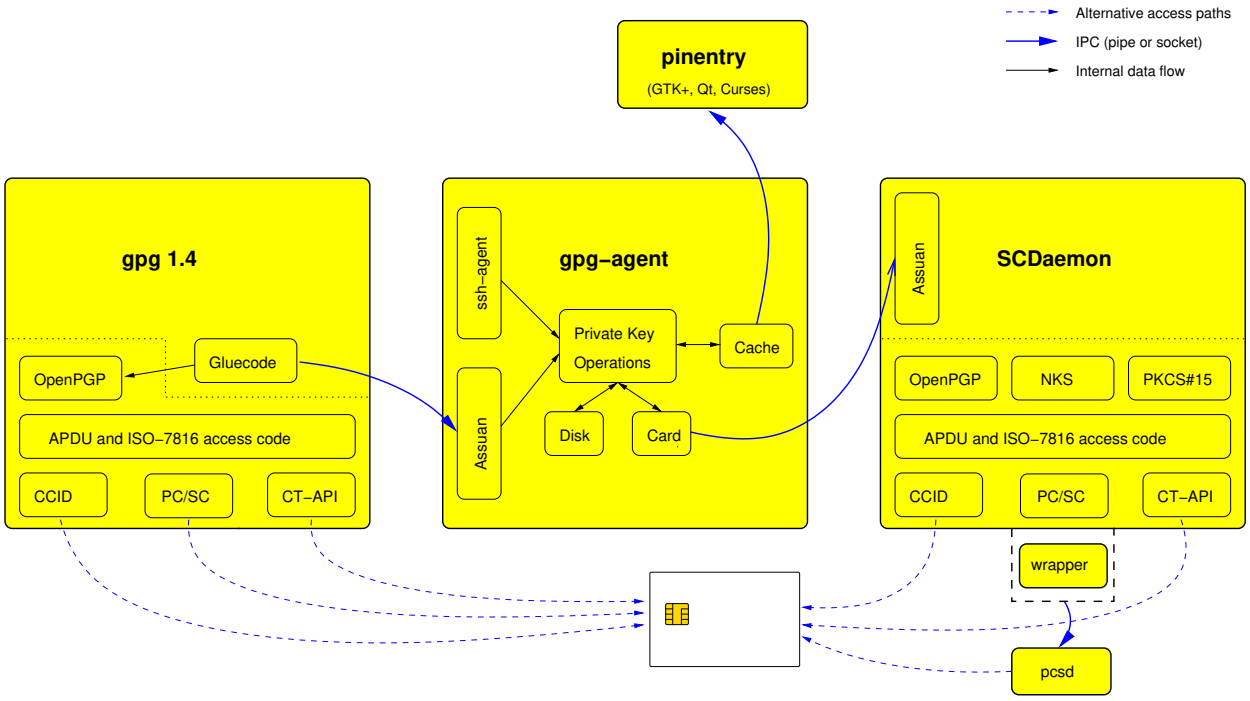
Az 5.1. ábra jobb oldali oszlopában vázlatosan az látható, hogyan épülnének fel az illesztések, ha az emuláció során kulcstároló eszköznek egy Android operációs rendszert futtató készüléket állítanánk be.

Az ADB eszköz lehetővé teszi nekünk többek között a telefonról futtatott szerver elérését

portátirányítás útján. Mint ahogyan a kérdőjel nyíl is jelzi, a megoldandó probléma innentől az, hogy ezt a rendszert hogyan illesztjük a GnuPG ökoszisztémához.

5.3. Lehetséges kommunikációs illesztési szintek emulációhoz

Ahhoz, hogy ebben a kérdésben döntést tudjunk hozni, először megvizsgálunk egy áttekintést arról, hogy az eredeti GnuPG milyen illesztéseken keresztül tud hozzáférni a kártyákhoz. Ebben nagy segítségünkre volt a 5.2. ábra a program kártyakezelő architektúrájáról, ami a forráskódjához volt mellékelve.



5.2. ábra. GnuPG intelligens kártya architektúra [11]

Azt látjuk, illetve, a forráskódot és a dokumentációt tanulmányozva azt tapasztaltuk, hogy a GnuPG egy Assuan nevű programkönyvtárat használ arra a célra, hogy az önálló folyamatként üzemelő komponensei között kapcsolatot teremtsen. Ez utóbbi modul tehát két folyamat közötti standardizált és biztonságos IPC kommunikációt biztosítja.

Az Assuan forráskódját megvizsgálva megállapítottuk, hogy csak a már Git tárolóban található első bejegyzése (a továbbiakban: commit) 2001-et szerepeltet a dátumában[8], amit a commit üzenet alapján Subversion (SVN) tárolóból emeltek át. Erre csak azért hívjuk fel

[8] `https://github.com/gpg/libassuan`

külön a figyelmet, mert érdekesnek találjuk, hogy a GnuPG - már-már azt mondhatnánk, korszakát megelőzve - nagyon hasonló architektúrában szerveződött több rétegen keresztül, mint a napjainkban egyre inkább elterjedő úgynevezett microservice alapú architektúrák. A mai Service Oriented Architecture (SOA) alapú programok szintén több különböző, egymástól függetlenül futó modulokból épülnek fel, amik egymással kommunikálnak működésük során.

Az 5.2. ábrán látható modulok közül az `scdaemon` az a program, ami a kártyákat kezeli, azok jelenlétét kommunikálja a `gpg-agent`, és azon keresztül a `gpg` program, illetve a `gpgsm` API implementáció irányába (utóbbi nincsen feltüntetve).

Több lehetőségünk van tehát arra, hogy megteremtsük a 5.1. ábráról hiányzó illesztést, azaz a rendszert „átverjük” egy virtuális kártyával. Ezek alacsonyabbtól a magasabb szintig:

- Elindulhatunk egészen onnan, hogy egy virtuális USB eszközt emulálunk, ami megvalósítja a Chip Card Interface Device (CCID) protokollt.
- Írhatunk egy olyan PC/SC implementációt, ami olyan válaszokat ad egy fiktív intelligens kártyától, ami megvalósítja az OpenPGP Smart Card szabványt [22]. Az írás pillanatában, a PC/SC-Lite forráskódjának vizsgálata során nem derült ki egyértelműen még számunkra, hogy utóbbihoz lehetséges -e egyéni illesztőprogramot írni, például egy megosztott függvénykönyvtár formájában, vagy másik `pcscd` implementációra van szükség ilyenkor.
- Megkísérelhetnénk ugyanezt a `CT-API` szabvánnyal is, de utóbbiról sajnos még kevesebb információt találtunk, és jelenlegi ismereteink szerint kevésbé elterjedt, mint a korábban említettek.
- Módosíthatjuk az `scdaemon` programot magát úgy, hogy a rendszer többi komponense felé jelentsen, és valahogyan működtessen egy kulcstároló eszközt, úgy, mintha az intelligens kártya lenne.
  - Külön kedvez ennek a megoldási lehetőségnek az a körülmény, hogy az `scdaemon` felülete már elfedi a kártya-specifikus tulajdonságait, és általános kezelési lehetőségeket kínál a rendszer többi komponensei felé.

5.3.1. Egy létező megvalósítás a PC/SC illesztés szintjén

Az írás pillanatában tudomásunk volt egy olyan, a közösség számára nyílt forráskódú projektről, ami az emulációt az alacsonyabb PC/SC szinten valósítja meg [20]. Ez utóbbi az

NFC modullal ellátott Android eszközöket célozza meg. Az eszköz az NFC kommunikációs interfészén elérhetővé teszi egy intelligens kártya által is megvalósított funkciókat. Ennek segítségével a telefon például bankkártya helyett használható.

A projekt az utóbbi konkrét alkalmazáson túl több alacsony szintű kommunikációs protokoll megvalósítását is tartalmazza a kliens oldalon - azon illesztőrétegét is, amit az OpenPGP Smart Card szabvány definiál [22].

A végpont oldalán történő emuláció felépítésekor talán ezt is fel tudnánk használni, ez azonban maga után vonná, hogy egy kártyaolvasó illesztőprogramot is el kell készítenünk. A forráskód első felületes tanulmányozása után nem voltunk benne biztosak, mennyire könnyen fogjuk tudni átemelni megvalósítás számunkra releváns elemeit a mi projektünkbe. Az emulátor jelenlegi kezdeti állapotában tehát kockázatosabb megoldási lehetőséggel kínálkozott a számunkra, mint az alternatívák, ezért egyelőre ezen az úton nem vizsgálódtunk tovább. [10]

5.4. Emuláció az `scdaemon` program szintjén

Mivel számunkra elegendő, hogy a kulcstároló a keretrendszer felől intelligens kártyaként viselkedjen, ezért arra jutottunk, hogy csak az ehhez feltétlenül szükséges meglévő rétegek viselkedését implementáljuk, legalább is kezdetben. A 5.2. ábrán ez maga az `scdaemon` program kiegészítését jelentheti. A későbbiekben alternatív illesztések implementálását - akár az 5.3.1. pontban említett projekt további tanulmányozásával - ugyanakkor nem gondoljuk elvetendőnek, hiszen ezek akár áthidalhatnak nem várt kompatibilitási problémákat.

Ha az `scdaemon` programot módosítanánk, az egyben azt is jelentené, hogy PC/SC szinten virtuális intelligens kártyára és virtuális kártyaolvasóra nem lenne szükségünk, és ezáltal az ezen rétegek között megvalósított protokollokra sem - ehelyett az `scdaemon` és a GnuPG többi komponensei közötti kommunikációs szabályokra kellene irányítanunk a figyelmünket.

5.4.1. A GnuPG beszerzése és fordítása

Különösebben nincsen nehéz feladatunk, hiszen a GnuPG projekt honlapján [11] a forráskódok fellelhetőek.

A projekt központi eleméhez három közvetlen könyvtárfüggés is tartozik. Ezek ugyancsak letölthetőek az előbbi forrásból. A mi a munkánkat a 5.1. táblázatban foglalt verziókra alapoztuk.

Minden komponens a GNU Autotools projektszervező eszközeivel fordítható le. A GNU/Li-

Szoftverkomponens	Verzió
GnuPG	2.2.20
`libgcrypt`	1.8.6
`libgpg-error`	1.39
`libassuan`	2.5.3

5.1. táblázat. Az általunk használt GnuPG komponensek verziói

nux ökoszisztéma előnyt jelent, de elméletileg a fordítás akár Cygwin, vagy MSYS2 / MinGW környezetben is lehetséges.

A fordítás az 5.1. táblázatban szerepeltetett minden projekt esetében a GNU/Linux rendszereken jártasabb felhasználók számára megszokott módon lehetséges. Itt most feltételezzük, hogy a `/usr/local/` könyvtár alá telepítjük a komponenseket.

```
./configure --prefix=/usr/local --enable-maintainer-mode
make
make install
```

5.1. listázás. A GnuPG komponenseinek fordítása

Az 5.1. listázásban szereplő utasítások rendre felderítik a fejlesztői operációs rendszer komponenseit, lefordítják, majd feltelepítik a programot. Ha valamilyen függőség - akár a fordítóprogram, akár egy alacsonyabb szintű könyvtár - nem áll rendelkezésre, akkor a `configure` szkript ideális esetben jelezni fogja a problémát. A függőség pótlása után ez újra lefuttatandó, egészen addig, amíg a folyamat hiba nélkül lejátszódik és létrejön a forráskönyvtárban a `Makefile`. Ezt a `make` parancs kiadásával lehet feldolgozni, minek hatására a program tárgykódja előáll az adott célkörnyezetben. Ha utóbbi művelet sikeres, akkor az `install` feladat elhelyezi az elkészült tárgykódokat és egyéb fájlokat a `--prefix` kapcsolóval megadott könyvtárban a Filesystem Hierarchy Standard (FHS) elvárásoknak megfelelően.

Az előbbieknek megfelelő rendre lefordítjuk a `libgpg-error`, a `libgcrypt`, `libassuan` könyvtárakat, és végül a GnuPG programot.

5.4.2. Egy Assaun modul működése

Az `scd` almappán belül megtaláljuk az `scdaemon` futtatható állományt.

Ezt éles környezetben a `gpg-agent` hozza működésbe, de manuálisan is elindítható, ha a

`--server` kapcsolót adjuk meg neki. Ahogyan a K.14. és K.15. listázásban is látható, a program dokumentálja magát - az elérhető parancsokat nemcsak felsorolja, de mindegyikhez rendelkezésre bocsát súgószöveget is. Ez alkalommal és a későbbiekben is, helymegtakarítás végett, ahol az 5.4.2. pontban szereplő parancsok előfordulnak, ott mindig csak azok egy részét fogjuk közölni.

Az F.1. listázásban azt látjuk, hogy egy tömböt definiálunk olyan `struct` adattípusokból, aminek a második eleme `assuan_handler_t` típusú. Minimális kutatás után meg is találjuk, hogy ez egy mutató egy

```
gpg_error_t assuan_handler (assuan_context_t ctx, char *line);
```

szignatúrájú függvényre. Mint azt megsejtettük és ellenőrzés útján kiderítettük, egy ilyen tömbelem egy parancsot definiál a K.14. listázásban látottak közül.

Érdemes megvizsgálni egy ilyen parancs implementációját. Az F.3. listázásban a `cmd_learn` függvénynek egy nagyon kis részlete látható - amiben ettől függetlenül minden fontosabb, a működéséhez szükséges építőelem fel lett tüntetve.

Egyrészt, figyelmesek lehetünk egy `assuan_context_t` példányra, ez a függvény kapcsolata a külvilág, azaz az `scdaemon` használója felé. Másrészt, a `line` argumentum a parancs után közölt argumentumlistát tartalmazza karakterlánc formájában. Aminek itt egészen pontosan tanúi lehetünk, az a kártya szoftverének illesztése, hiszen itt látunk APDU kommunikációra vonatkozó utasításokat. Megállapíthatjuk azt is, hogy a program többféle kártya kezelésére nincs felkészülve, kizárólag az OpenPGP Smart Card szabványt [22] megvalósító eszközökre áll készen.

Mivel az `scdaemon` szintjén történő emuláció mellett döntöttünk, ezért a következő lépés, hogy elérjük, hogy az előbbi komponens APDU utasítások helyett egy fiktív intelligens kártyát működtessen, ami mögé később egy kulcstároló eszközt implementálhatunk. Nem szerettük volna, ha a módosításaink elvennének funkciókat a meglévő programból, ezért az `scdaemon`-t úgy módosítottuk, hogy lehetőséget adjon az alább felsorolt kártyára vonatkozó parancsok egyéni implementációinak betöltésére.

Ezekről a parancsokról gondoljuk azt, hogy nem általában az `scdaemon` programra vagy a keretrendszerre, hanem kifejezetten a kártyára vonatkoznak, azaz kártyától függ az implementációjuk: `SERIALNO`, `LEARN`, `READCERT`, `READKEY`, `SETDATA`, `PKSIGN`, `PKAUTH`, `PKDECRYPT`, `GETATTR`, `SETATTR`, `WRITECERT`, `WRITEKEY`, `GENKEY`, `RANDOM`, `PASSWD`, `CHECKPIN`, `DISCONNECT`, `APDU`, `GETINFO`.

5.4.3. Egyéni kártyaimplementáció betöltése

Az ezen pont címében szereplő funkció az eredeti programnak nem része, így ezzel mi egészítettük azt ki. A teljes implementációt - ismételten helytakarékosság céljából - nem ismertetjük, azonban a dolgozatban hivatkozott forráskódot a későbbiekben elérhetővé fogjuk tenni az egyik szerző weboldalán[9].

A program indítási beállításai közé felvettük a K.16. listázásban olvasható `--custom-backend` elemet. Ezzel dinamikusan lehet betölteni olyan programkönyvtárat, ami az F.2. listázásban szereplő fejlécállományt implementálja.

Ennek kidolgozásakor nehézséget okozott, hogy a program csak akkor töltődött be a legnagyobb biztonsággal, ha nem állítottunk elő olyan állapotot, amelynek során könyvtárverziók ütköztek egymással. Személyes tapasztalatunk, hogy alacsony szintű programozásnál általában egyszerűbb nagyon óvatosnak lenni, mint hibát keresni.

Sajnos, ha ütközés, vagy egyéb nehezen megérthető akadály állt elő, akkor a dinamikusan fordított könyvtárak futás közbeni betöltéséért felelős `dlopen()` függvény egyszerűen hibát jelző értékkel tért vissza (`NULL`).

Az előbbi helyzetet orvosolni ott lehet problémás, hogy a betöltendő dinamikus könyvtár már pusztán a `gpg_error_t` és az `assuan_context_t` típusok jelenléte okán függ a `libassuan` és a `libgpg-error` könyvtáraktól - nem beszélve az olyan elkerülhetetlenül meghívandó függvényektől, mint az `assuan_write_status()`, amiknek egyeztetését valahogyan meg kell oldani az `scdaemon`-hoz fordított könyvtárakhoz.

A hiba keresése során legnehezebben felfedezhető ütközések egyike éppen abban az esetben áll elő, amikor a rendszerben telepített GnuPG verziója összeakad a dinamikusan betöltendő `scdaemon` modulnak egy saját függésével - például más verziójú vagy fordítású `libassuan` könyvtárral építették a modult, mint amilyennel magát az `scdaemon` programot.

5.4.4. A könyvtárütközések elkerülése

Szerencsére, amíg minden komponensből egy van jelen a fejlesztői számítógépen, addig az 5.4.3. pontban említett nehézséggel még nem kell feltétlenül foglalkozni. A felállás akkor kezd el érdekessé válni, amikor...

1. ...elkezd a módosított projektünk összeakadni a rendszeren beépítetten feltelepített könyvtárakkal. A GNU/Linux alapú rendszerek, főképp a csomagkezelés terén rend-

[9] `https://www.codekuklin.com/`

szerint aktívan használják a GnuPG programot. Ha csak ebből a komponensből lefordítunk egy saját - adott esetben módosított - verziót, akkor a rendszeren ez még nem okoz problémát. Azt ugyanakkor már ezen a ponton is figyelembe kell venni, hogy a könyvtárfüggőségeiből ez a kézileg épített verzió a rendszeren megtalálható `libgpg-error`, `libassuan`, `libgcrypt` könyvtárakat fogja megkísérelni meghivatkozni és használni.

(a) Ez a körülmény akkor kezd igazán problémát jelenteni, amikor

i. Nem kompatibilis valamelyik jelenlévő könyvtárral a lefordított GnuPG, mert túl nagy a verzióeltérés. Ekkor a felhasznált könyvtárakat is újra kell fordítani.

ii. A programunk futásakor a hibakeresővel a meghivatkozott könyvtárak egyikének működését is meg szeretnénk vizsgálni - például a `libassuan` könyvtár működését. Ehhez hibakereső szimbólumokkal ellátott, illetve optimalizálások nélkül lefordított tárgykódra van szükség. Ez rendszerint nem érhető el a csomagkezelőből, tehát saját verziót kell lefordítani.

2. ...előtérbe kerül, hogy az ökoszisztéma egyes részeit egy másik CPU architektúrájú hardverre is le kell fordítanunk - a segédeszköz hardverére. Ilyen könyvtár lesz például a későbbiekben a `libgcrypt`, ami általános titkosításra, és az `scdaemon` által elvárt kommunikációs szabvány betartására használható eszközöket tartalmaz.

(a) Itt a valódi fennakadást többnyire nem a két különböző architektúrára lefordított tárgykód jelenti. A komplikáció akkor állhat elő, ha valamilyen működés a segédeszközön nem úgy valósul meg, ahogyan a fejlesztői számítógépen. Ilyenkor sajnos a legkézenfekvőbb hibakereső eszközünk az, hogy megpróbáljuk közelíteni egymáshoz a két környezetet. Első lépésnek tehát nem árt a megcélzott könyvtárverziót a saját fejlesztői rendszerre is újra lefordítani (a csomagkezelőből telepített verzió helyett), hiszen itt még viszonylag könnyen tudjuk használni a hibakereső (debugger) eszközöket, ellenben például az ARM processzoros telefonnal. Ekkor megint csak megjelent a rendszerünkön a második verzió ugyanabból a könyvtárból.

Miből állapítja meg a fejlesztői gép, hogy futtatáskor melyik könyvtárverziót kell használni?

A GNU/Linux platformon fordításkor és futtatáskor megadható, erre vonatkozó beállításokat az 5.2. táblázatban foglaltuk össze.

Fordítási időben a `-L` kapcsolóval állíthatjuk be (például `-L/path/to/libs`), hogy a fordító melyik fájlrendszeri mappában keresse a könyvtárakat, amelyek közül a linkernek a `-l` kapcsolóval adhatjuk meg a kiválasztottak listáját (például `-lfoo`).

A program indításakor a könyvtárak hollétéről két iránymutatást tudunk adni:

- Az ELF állományokban az úgynevezett Run-time search path (rpath) fejlécadat állításával, amibe bele lehet írni fájlrendszeri könyvtárakat, ahol a `lib` fájlnévkezdetű keresett állomány megtalálható.
- Ugyanilyen keresési útvonalakat adhatunk meg az `LD_LIBRARY_PATH` környezeti változóban is - ez utóbbi nagyon rugalmasan teszi lehetővé helyzettől függően, hogy milyen könyvtárakat ajánlunk az operációs rendszer figyelmébe priorizálásra.

A kettő közül tapasztalatunk szerint az rpath a ritkábban alkalmazott megoldás, vélhetőleg mert a legtöbb disztribúción feltételezett, hogy a dinamikus függvénykönyvtárak az FHS-nek megfelelő szerkezetben vannak elhelyezve - illetve, hogy minden telepített komponens pontosan ugyanattól a verziótól függ. A megfelelő fordítói kapcsolókkal az rpath állítható manuálisan, és így el tudtuk érni a fejlesztésre használt gépeken, hogy a fejlesztői könyvtárak és a GnuPG programok ne zavarják egymás működését.

	Kategória	Beállítás	Megjegyzése
Fordítási idő	Fordítói kapcsoló	`-L`	Könyvtárak keresési útja
	Fordítói (linker) kapcsoló	`-lfoo`	`foo` könyvtár linkelése
Futtatási idő	Környezeti változó	`LD_LIBRARY_PATH`	Könyvtárak keresési útja
	ELF fejlécadat	`RPATH`	Könyvtárak keresési útja

5.2. táblázat. A könyvtárak meghivatkozása fordítási és futási időben, GNU/Linux platformon

5.5. Látszólagos intelligens kártya implementáció

Az 5.4.3. pontban ismertetett funkció birtokában készen állunk rá, hogy elkészítsünk egy olyan `scdaemon` modult, ami elviekben akár a teljes az eredetileg tervezett teljes működést is meg tudná valósítani. Első lépésben azonban egy egyszerűbb implementációt készítünk, ami segít feltérképezni azt, hogy a GnuPG milyen viselkedésre számít az `scdaemon` részéről.

5.5.1. noop implementáció

Kezdetben a legnagyobb egyszerűséggel egy olyan modult hoztunk létre, ami a parancsok és annak argumentumainak naplózásán túl nem tesz egyebet. Ennek megvalósítását ismerteti az F.4. listázás. A naplózás pontos implementációját nem szerepeltetjük, azt ugyanakkor érdemesnek tartjuk megjegyezni, hogy olyan megoldást választottunk, amivel nagyon egyszerűen bármikor meg tudjuk változtatni annak a kimeneti csatornáját. Az írás pillanatában a fejlesztői számítógép syslog alrendszerébe irányítottuk a kimentet:

```
okt 14 15:03:22 ceres kc-syslog[472507]: custom_cmd_learn(), args: --keypairinfo
```

A fejlesztés során ez a modul is segítségünkre volt abban, hogy megállapítsuk, hogy a GnuPG különböző parancssori hívásainak hatására az `scdaemon` milyen parancsokat fogad. Ez az eszköz egészen odáig hasznosnak is bizonyult, amíg meg nem állapítottuk, hogy a GnuPG rendelkezésre bocsát fejlesztőknek olyan hibakereső eszközt, ami rendkívül átláthatóan felfedi a teljes kommunikációt (K.17. listázás).

5.5.2. Statikus implementáció

Következő célnak azt tűztük ki, hogy a kártya modulunkkal egy ismert fájlt sikeresen alá tudjunk írni egy ismert kulccsal. Nem volt tehát célunk, hogy valódi aláírás jöjjön létre tetszőleges állományra, csupán az, hogy az ehhez szükséges kommunikáció az `scdaemon`-nal megvalósuljon, és végeredményben előálljon egy (előre bekészített) érvényes aláírás. Ennek kipróbálásához tehát létrehoztunk egy minta kulcsot, egy minta állományt és egy érvényes aláírást arra, helyileg.

A művelet után folytattuk a GnuPG és a `scdaemon` viszonyának tanulmányozását. Ennek során vizsgáltuk az `scdaemon` eredeti rutinjait, a másik oldalról a `gpg` kliens forráskódját és a GnuPG kézikönyvének [11] `scdaemon` parancsait leíró fejezeteit.

Megállapítottuk, hogy a `gpg --card-status` parancs az információkért a `LEARN` paranccsal fordul kártyakezelő programhoz, és különböző tulajdonságokat leíró úgynevezett „status line”-okra számít, amiből például kiderül a kártya tulajdonosa, sorozatszáma, a rajta futó szoftver, és különböző információk a kulcsokról (K.18. listázás).

Ahhoz, hogy a `gpg` kliens program érzékeljen egy kártyát, már a sorozatszámot is elegendőnek bizonyult a megfelelő formátumban visszaküldeni, amit az OpenPGP Smart Card szabvány [22] definiál. Valójában az `scdaemon` nem a kártya sorozatszámával, hanem az előbbi

szabványban definiált úgynevezett Application Identifier (AID) bájtsorozattal tér vissza. A statikus implementációnál a mi esetünkben ez az 5.3. táblázatban foglaltak szerint épül fel.

`D276000124010340FFFF123456780000`

Bájtsorozat	Értelmezés	Mező
`D276000124`	Free Software Foundation, Inc.	ISO 7816-5 szerinti alkalmazás szolgáltató
`01`	OpenPGP alkalmazás	Alkalmazás azonosító
`0340`	3.4	Alkalmazás verzió
`FFFF`	Speciális, „tesztelés”	Gyártó azonosító
`12345678`	-	A kártya sorozatszáma
`0000`	-	Fenntartott

5.3. táblázat. A statikus kártyamodul AID azonosítójának felépítése

Ezt a sorozatszámot az Assaun segítségével felküldtük a `gpg` kliensnek (F.5. listázás), mire az sikeresen észlelt egy kártyát (K.19. listázás).

Ahhoz, hogy a `gpg` kliens nyilvántartásba vegye a virtuális kártyán jelenlévő nyilvános kulcsot, még jelenteni kell néhány információt. Egyrészt, szükség lesz a nyilvános kulcs ujjlenyomataira, másrészt a titkosítási algoritmus paramétereit kódoló úgynevezett Canonical S-expression (csexp) kifejezésekre. Első kísérletünkben egy RSA nyilvános kulcs paramétereivel tértünk vissza.

5.6. A nyilvános kulcs kommunikálása a gpg kliensnek

Kigyűjtöttük azokat az adatokat az 5.5.2. pontban létrehozott kulcsról, amikkel a statikus modulból vissza szeretnénk térni.

5.6.1. Kulcs ujjlenyomat

Az ujjlenyomat kiolvasása egyszerű feladat volt, hiszen a `gpg` minden további nélkül elárulta ezt az információt, ha megfelelő argumentumokkal hívtuk. Az ugyanakkor meglepetésként ért minket, hogy nem csak egyféle ujjlenyomatról számol be a kézikönyvben [29] leírt protokoll szerint az `scdaemon`. Utóbbi megkülönböztet úgynevezett „fingerprint” lenyomatot - amivel gyakrabban találkozunk a program használata során - és „key grip” lenyomatot. A „key grip” számításakor a rendszer nem veszi figyelembe a kulcs GnuPG-specifikus tulajdonságait, csak az adott kulcsalgoritmus paramétereit.

A `LEARN` parancs implementációjában visszaadtuk ezeket az értékeket (F.8. listázás) - ez azonban még nem volt elegendő ahhoz, hogy a `gpg` érzékelje a nyilvános kulcs jelenlétét - ilyenkor ugyanis, ha az iménti listázásban látható módon egy `KEYPAIRINFO` status line útján jelentünk egy kulcsot, akkor a `gpg` mindjárt le is kérdezi a nyilvános paramétereket - és ezek hiányában hibát jelez.

A nyilvános paraméterek felküldését az nehezítette, hogy ezeknek mindenekelőtt birtokában kellett lennünk. Ahhoz, hogy az eddig küldött adatok összhangban legyenek, a meglévő kulcs paramétereire volt szükségünk. Ezek lekérdezésére sajnos nem állt rendelkezésre közvetlen API felület (a `gpgme` nem bocsátott rendelkezésre ilyen függvényeket), ezért, a statikus implementációhoz, kézileg olvastuk ki egy szerializált kulcsfájl tartalmát a `libgcrypt` könyvtár segítségével.

5.6.2. RSA paraméterek olvasása - OpenPGP Message Format

Egy RSA [24] titkos kulcsot - mellőzve a most következők pontos szerepének ismertetését - hat számszerű paraméter határoz meg, melyből kettő formálja az absztrakt nyilvános kulcsot. Azért, hogy a nyilvános paraméterekből a titkos paraméterek visszafejtése közelítőleg lehetetlen nagy kihívást jelentsen a számítógépeknek, ezek a számok rendkívül nagy kiterjedésű, úgynevezett multiple-precision integer (MPI) értékek.

A bináris formátumban szerializált kulcs felépítését az OpenPGP Message Format szabvány definiálja [7]. Utóbbi terminológia szerint a fájl egy több „csomagból” (packet) felépülő OpenPGP üzenetet tartalmaz. A titkos kulcsot leíró csomag felépítésének egy részlete az 5.4. táblázatban olvasható, a teljes specifikáció nyomán.

Mező	Hossz	Érték / megjegyzés
Header (PTag)	1 bájt	Secret-key packet jelzés
Packet hossz	2 bájt	920 oktet
Verzió	1 bájt	4
Létrehozás dátuma	4 bájt	2020. szeptember 8. 15:19:10 GMT+2
Algoritmus	1 bájt	RSA
n MPI	2048 bit	-
...		

5.4. táblázat. Részlet a secret-key packet-ből kiolvasható információkból, a próba kulcsra vetítve

A mi esetünkben egy titkos kulcsból nyertük ki a nyilvános paramétereket. Ehhez meg

kellett találni a titkos kulcsot leíró csomagot, és a szabványt követve beolvasni a tartalmát a memóriába. Az MPI számok kezeléséhez segítségül hívtuk a `libgcrypt` könyvtárat. A K.20. listázásban az RSA paramétereket kiolvasó modulunk naplókimenete látható.

5.6.3. Nyilvános RSA paraméterek felküldése S-expression kifejezések formájában

A `gpg` kliens naplókimenetét tanulmányozva, illetve további debug futtatás után megállapítottuk, hogy az 5.6.1. pontban is említetten miért kaptunk hibajelzést a kulcs lenyomatainak felküldése után - ez pedig az, hogy a `gpg` a kulcs lenyomatainak olvasása után a `READKEY` paranccsal folytatja a működését, arra várva, hogy válaszul visszatérünk a kulcs nyilvános paramétereivel egy csexp kifejezés formájában. Bár az `scdaemon` protokolljának ismertetésekor ez a GnuPG kézikönyvben is le van írva, ott azonban ennek részletei nincsenek különösebben kifejtve:

> Return the public key for the given cert or key ID as an standard S-Expression. [29]

Figyelmesek lettünk ugyanakkor arra, hogy a `libgcrypt` kézikönyve [27] definiál S-expression (sexp) kifejezéseket RSA [24], DSA [21] és ECC [19] [14] kulcsokra. A kézikönyv nyomán a próba kulcsból kiolvasott adatokból megalkottuk az 5.2. listázásban szereplő formátumú csexp kifejezést. Ezt felküldtük az F.9. listázásban szereplő módon. Nagy örömünkre, a `gpg` kliens érzékelte a kulcsot, és megjelenítette a kártya adatai alatt (K.23. listázás).

Ezek szerint valóban a `libgcrypt` által előírt sexp formájú paraméterekre volt szükség.

```
(public-key
    (rsa
        (n n-mpi)
        (e e-mpi)))
```

5.2. listázás. Publikus RSA paraméterek a `libgcrypt` sexp formátumában

Ebben a dolgozatban az emuláció terén idáig sikerült eljutnunk. Ezzel párhuzamosan megvizsgáltuk, hogy milyen módon lehetséges a kis teljesítményű, többnyire ARM processzor architektúrájú eszközökön C nyelven íródott függvénykönyvtárakat használni, mind kézzel indítva, mind egy Android alkalmazás által hívottan - utóbbi esetben a Java Native Interface (JNI) technológia felhasználásával.

6. Kulcstároló eszköz megvalósítása korai Android eszközön

A 3.3.4. pontban megállapítottuk, hogy korai, már nem használt, Android operációs rendszerrel ellátott okostelefonjaink alkalmasak lehetnek kulcstároló eszközként való használatra. Szeretnénk megragadni az alkalmat itt is, hogy visszautaljunk a 4. pontban foglaltakra, miszerint a projektbeli kulcstároló eszköz nem feltételez okostelefont - ez csak az egyik a lehetséges implementációk között.

A fentiek közül azért választottuk az Android okostelefonok programozását, mert arra számítunk, hogy az idő előrehaladtával egyre több, erre a célra felhasználható, egyébként kihasználatlan eszköz fog rendelkezésre állni, többek között az érdeklődők háztartásában is.

Az általunk összeállított, és a továbbiakban tesztelés és alkalmazásfejlesztés során igénybe vett, Android 1.5 operációs rendszert támogató fejlesztői rendszer összetételéről az A. mellékletben olvasható egy rövid leírás.

6.1. Kapcsolattartás a kulcstároló eszközzel

A 3.3.4. pontban a kulcstároló eszközzel szemben azt az elvárást fogalmaztuk meg, hogy képes legyen kapcsolatot tartani egyfelől a titkosítási műveletek elvégzésekor a végponttal, (opcionálisan) másfelől - elsősorban megerősítés és közvetlen konfiguráció céljából - saját perifériáin keresztül a felhasználóval.

A továbbiakban azt vizsgáljuk, hogy alacsony szintű könyvtárakkal hogyan valósítható meg a kommunikáció a végpont és a kulcstároló eszköz között.

6.1.1. Kommunikációs protokoll

A kapcsolattartás módja eredeti elképzelés szerint szöveges alapú HTTP feletti JSON üzeneteket értelmező és válaszoló REST felület lett volna. A jelenlegi elgondolásunkban azonban már nem vagyunk benne teljesen biztosak, hogy eredendően bináris adatokat célszerű -e a felületen való átküldés előtt szöveges formájúvá alakítani, majd átküldés után visszaalakítani. Végső soron tehát könnyen elképzelhető, hogy eltekintünk a JSON formátumtól, és egyáltalán a szöveges kommunikációs formától. Helyette esetleg az 5.6.3 pontban megismert, bináris adatok továbbítására talán alkalmasabb csexp kifejezésekkel válthatnánk üzeneteket, megtartva a HTTP protokollt.

A 6.2. pontban tárgyaltak még a projekt kezdeti fázisában készültek el, ezért ott egy REST felület kezdetleges megvalósítása olvasható alacsony szintű könyvtárakkal, Android készüléken. Bár nem teljesen biztos, hogy végül éppen ebben a formában valósítjuk meg a kommunikációt, mindazonáltal releváns marad, hogy hogyan lehetséges Linux alapú rendszereken használható könyvtárakat más processzor architektúrájú és más szerveződésű környezetben felhasználni.

6.2. Függvénykönyvtárak fordítása ARM processzoros Android eszközre

Kiindulásképpen fizikai Android eszköz helyett az emulátorban futtatható legkorábbi - Android 1.5 - operációs rendszerrel ellátott virtuális eszközön tettünk kísérletet arra, hogy alacsony szintű könyvtárak segítségével egy célnak megfelelő kiszolgáló programot futtassunk.

Az igen korai Android verzióra alapozó törekvéseinket arra vonatkozó bizakodásunkkal indokoljuk, hogy a jövőben kiadott futtatási környezetek - a későbbi Android készülékek - a megoldásunk natív tárgykódú részével továbbra is kompatibilisek maradnak, legkevesebb olyan értelemben, hogy módosítás nélkül lefordítható marad a forráskód az új architektúrára.

Az emulált processzor néhány lekérdezhető jellemzője a K.24. listázásban látható.

A kérdéses operációs rendszerhez rendelkezésére bocsátott SDK tartalmazta a Linuxon használatos `gcc` és `clang` fordító-, és az `ld` linker programokat, valamint a Linux ökoszisztémában megszokott alacsony szintű platform-specifikus implementációjú fejlesztői eszközöket.

Saját, néhány állományból álló programok lefordítása során nem ütköztünk nehézségekbe. Ezeket az állományokat fel tudtuk tölteni a virtuális eszközre és a megfelelő könyvtárból sikeresen tudtuk futtatni azokat (K.25. listázás).

Jelentősebb mértékben okozott ugyanakkor bonyodalmat olyan programok lefordítása, amelyek már valamilyen könyvtárfüggéssel bírtak. Ilyen volt például a tesztelési célból elkészített kezdetleges REST felület implementáció, ami máskülönben nem tett egyebet, minthogy egy adott végponton visszaadott egye előre beállított teszt választ.

Ehhez már szükség volt rá, hogy lefordítsuk a használni kívánt könyvtárakat az adott architektúrára. Hibakeresés céljából indokolt volt pontosan ugyanezen könyvtárak lefordítása a fejlesztői platformra is, a fordítói optimalizálások kikapcsolásával, és a keletkezett tárgykódban a hibakereső szimbólumok meghagyásával. Utóbbi törekvésünk sajnos azzal a mellékhatással járt, hogy bizonyos - olykor az `scdaemon` kártyaimplementációk (5.4.3. pont) által

is használt - könyvtárak a fejlesztői operációs rendszeren több példányban is megjelentek, ami a program betöltésekor a fejlesztő számára ambivalens állapotot idézett elő. Ahhoz, hogy egyértelművé váljon, hogy fordítási és futtatási időben a program melyik könyvtárakat hivatkozza meg, az 5.4.4. pontban közlünk megoldási lehetőségeket.

További kérdéseket vetett fel, hogy a legtöbb esetben a felhasznált könyvtárak maguk is más könyvtárakra alapozzák funkcionalitásukat, ami azzal a következménnyel jár, hogy használatukhoz a függőségeiket is elérhetővé kell tennünk fordítási, és esetenként futtatási időben is. Az erre vonatkozó, GNU/Linux platformon fordítási és futási időben megadható beállításokat az 5.2. táblázatban foglaltuk össze, míg az ezzel kapcsolatos további komplikációkat és az orvoslás választott módját a 6.2.4. pontban tárgyaljuk.

6.2.1. Autotools projektek fordítása

A felhasznált könyvtárak forráskódjának többsége GNU Autotools rendszerben szerveződtek. Általában ezeket a projekteket a főkönyvtárban elhelyezkedő `configure` parancsállomány jelenlétéből lehet felismerni. Ez utóbbi szkript feladata, hogy fordítás előtt felderítse a célplatformot, ellenőrizze, hogy rendelkezésre állnak -e a fordítói eszközök, és előkészítse a fordítási egységek egymásra épüléseit leíró `Makefile` állományt.

A szkript futása után - amennyiben az sikeres volt - a fordítási folyamat a `make` programmal indítható, ami az egyes forrásállományokat a függési fa figyelembevételével topologikus sorrendbe rendezi, és rendre lefordítja - igény szerint automatikus párhuzamosítás alkalmazásával. Ezekkel az utasításokkal fordítottuk az 5.1. listázásban a GnuPG rendszer szoftvereit is.

Egy C nyelven íródott, GNU Autotools rendszerben szerveződő könyvtár Android platformra való lefordításához a következőkre volt szükség:

1. A célplatform alacsony szintű fejlesztői eszközei a keresési útvonalon (`PATH` környezeti változóban foglalt könyvtárak) kellett, hogy szerepeljenek, a megfelelő platform azonosító előtagjával ellátott névvel. Például, az Android SDK általunk célzott korai eszközökre szánt fejlesztői programjainak mindegyike az `arm-linux-androideabi-` előtaggal kezdődik, így a `gcc` fordító futtatható állományának neve `arm-linux-androideabi-gcc`, az `ld` linker program futtatható állományának neve `arm-linux-androideabi-ld`, és így tovább.

2. A `configure` állomány hívása előtt a `CFLAGS` környezeti változóhoz hozzá kellett fűzni az SDK-ban található alap fájlrendszer elérési útját, és hívásakor meg kellett adni a

célplatform azonosítóját - esetünkben a `--host=arm-linux-androideabi` argumentummal. (6.1. listázás).

```
export CFLAGS="$CFLAGS
↪  --sysroot=/opt/kuklinistvan/android-ndk-r10e/STANDALONE/arm/sysroot"
./configure --host=arm-linux-androideabi
```

6.1. listázás. GNU Autotools projekt konfigurálása Android rendszerre történő fordításra

6.2.2. CMake projektek fordítása

A CMake egy másik program ugyanannak a problémának a megoldására, mint amire a GNU Autotools eszközeit használjuk. Ahhoz, hogy egy CMake rendszerben szerveződő könyvtárat Android rendszert futtató eszközön felhasználhassunk, a GNU Autotools esetében tárgyaltakhoz képest valamivel egyszerűbb eljárást kell követnünk. Utóbbi legkevesebb annyiból áll, hogy annak konfigurálása során a meglévő argumentumok listáját a 6.2. listázásban olvasható példában szereplőkkel kiegészítjük.

```
/usr/bin/cmake \
    -DCMAKE_ANDROID_STANDALONE_TOOLCHAIN="/path/to/ndk-toolchain" \
    -DCMAKE_BUILD_TYPE=Release \
    -DCMAKE_SYSTEM_NAME=Android \
    -DCMAKE_SYSTEM_VERSION=3 \
    -DCMAKE_ANDROID_ARCH_ABI=armeabi \
    -DANDROID_ABI=armeabi \
    -DANDROID_PLATFORM=android-3 \
    <projekt könyvtár>
```

6.2. listázás. CMake projekt konfigurálása Android rendszerre történő fordításra

6.2.3. A sikeresen lefordított könyvtárak

Az írás pillanatában csupán a fenti két (tapasztalatunk szerint leggyakoribb) rendszerben szerveződő könyvtárakat használtunk fel. Ezeket kisebb-nagyobb finomhangolásokkal, beállítások módosításával, ritkán egyéni foltozás írásával, de sikerült lefordítani mind az Android, mind a fejlesztői operációs rendszerre.

Annak részleteit itt most nem ismertetjük, hogy az egyes könyvtárakat milyen további, specifikus lépésekkel sikerült integrálni, de terveink szerint ezeket a forrásokat elérhetővé fogjuk tenni az egyik szerző weboldalán[10]. Az írás pillanatában a csomagforrások szintén a szerzők egyikének GitHub oldalán érhetőek el[11]. A REST felületet implementáló könyvtárnak (Ulfius), és annak minden köztes függőségét a 6.1. táblázatban közöljük.

Néhány, az Android platformra történő tárgykód generálás során jellemzően fontosabb, a `gcc` program hívásakor átadott argumentum a K.21., `kc-core` komponensre vonatkozó fordítói argumentumok listázásában olvasható.

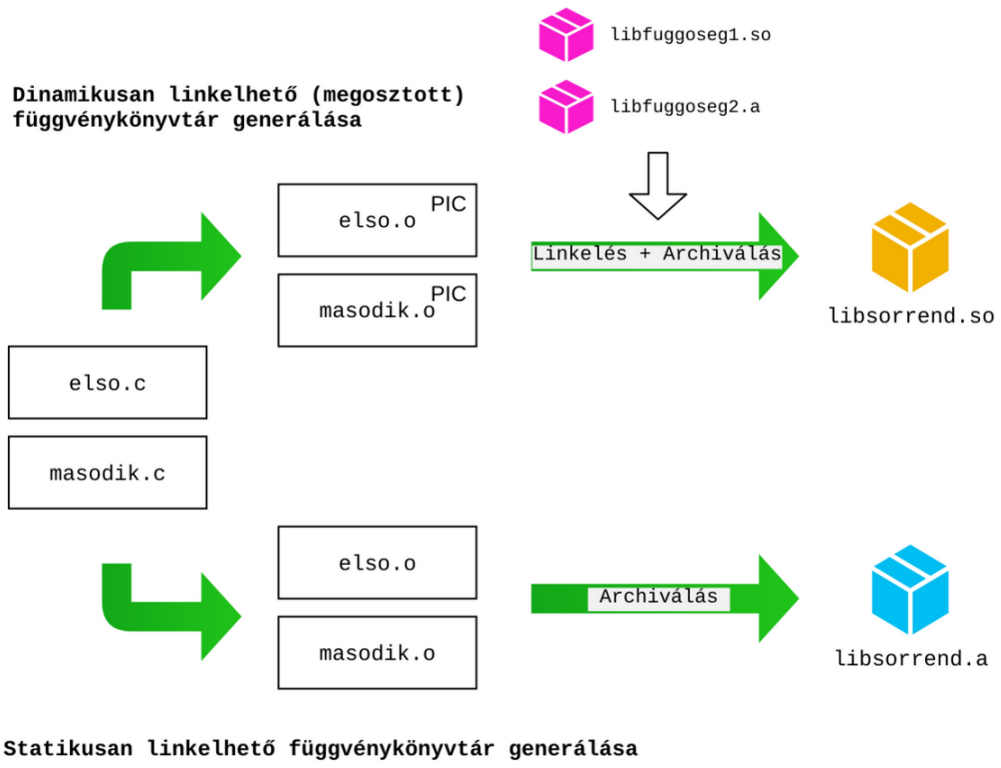
Programkönyvtár	Verzió
Ulfius	2.6.5
GnuTLS	3.6.12
`libgmp`	6.2.0
`nettle`	3.5.1
`libtasn1`	4.16.0
`libunistring`	0.9.10
`libmicrohttpd`	0.9.70
`jansson`	2.12
`libcurl`	7.71.1
`libidn2`	2.0.5

6.1. táblázat. A `kc-core` komponensben megvalósított REST felület lefordításához szükséges minden köztes, általunk felhasznált könyvtárfüggőség

6.2.4. Statikusan és dinamikusan linkelt könyvtárak használata

A harmadik féltől származó, C nyelven íródott függőségek egységbe zárásásakor - könyvtárak létrehozásakor - a fordítói ökoszisztéma lehetőséget ad arra, hogy utóbbiakat arra készítsük fel, hogy a programunkhoz azokat *dinamikusan*, vagy *statikusan* „kössük” a későbbiekben (az általunk is magyarosított eredeti terminológia szerint és a továbbiakban: „linkeljük”).

[10] `https://www.codekuklin.com/`
[11] `https://github.com/kuklinistvan/libraries-for-android-ndk-r10e`



6.1. ábra. Statikusan és dinamikusan hivatkozható (linkelhető) könyvtárak fordítása

A kétféle könyvtár generálása

Tekintsük felületesen egy könyvtár kétféle fordításának lehetséges lépéseit, hogy jobban megértsük, mi a különbség a kettő megközelítés között.

A tárgykód generálása során első lépésben lefordítjuk a forrásállományokat a megcélzott processzor architektúrának megfelelő gépi kódra (6.1. ábra). Fordítási egységenként ilyenkor létrejön egy-egy tipikusan `.o` kiterjesztésű objektum fájl, amik még önmagukban nem futtathatóak vagy használhatóak.

1. Ha statikusan linkelhető könyvtárat szerettünk volna előállítani, akkor a létrejött objektum állományokból archívumot generálunk, az iménti fordítói ökoszisztémában az `ar` program segítségével. Ezzel el is készült a - GNU/Linux platformon általában `.a` kiterjesztésű - könyvtár állomány.

2. Amennyiben ezekből az objektum fájlokból megosztott, dinamikusan linkelhető függvénykönyvtárakat szeretnénk előállítani, akkor már azok generálása során meg kell adnunk a `gcc` fordítónak az `-fPIC` kapcsolót, aminek hatására az előállítja az ún.

Position-independent code (PIC) tárgykódot[12]. Az így keletkezett objektum fájlokat utána:

(a) Linkeljük a saját függőségeihez.

(b) Létrehozzuk belőlük a futásidőbeni betöltéshez optimalizált megosztott objektum („shared object") állományt. Bár a 6.1. ábrán itt is „archiválás" címke szerepel, az ily módon előállt, tipikusan `.so` kiterjesztésű állomány az objektum állományok tartalmán felül a gyors és megbízható futásidejű betöltést segítő információkat is tartalmaz.

A kétféle könyvtár felhasználásának módja

Az így keletkezett könyvtárak a használatukban eltérnek:

- Egy statikusan linkelhető könyvtár releváns elemei a végső futtatható tárgykód előállításakor az utóbbi forrásaiból előállított objektum fájlokkal egy időben kerül linkelésre - úgy, mintha a statikus könyvtárat alkotó objektum állományokat egyszerűen hozzávettük volna a fordítási folyamathoz. Előnye ennek, hogy a linker képes a keletkezett futtatható tárgykód méretét optimalizálni úgy, hogy a statikus könyvtárnak csak a felhasznált elemeit építi be a keletkezett állományba. Az így keletkezett futtatható állomány a harmadik féltől származó kódot magában hordozza, mintha az a saját forráskódjából került volna bele. Megjegyezzük ugyanakkor, hogy a statikus könyvtár a saját függéseit általában nem szokta magában hordozni, így tehát a köztes függőségeknek szintén jelen kell lenniük a fordítási folyamat során.

- Dinamikusan linkelhető függvénykönyvtár esetén a tipikusan `.so` kiterjesztésű állomány tartalma nem fogja részét képezni a végső futtatható tárgykódnak. Bár ilyenkor arra nincsen lehetőség, hogy a könyvtár méretét a ténylegesen felhasznált funkcionalitás függvényében automatikusan optimalizáljuk, azonban egy ilyen `.so` állományt egy rendszerben, akár egy időben több program is használhat - ennélfogva hívjuk megosztottnak. Bizonyos körülmények között akár arra is lehetőség van, hogy ezen tárgykódot két futás között frissítsük, anélkül, hogy a ráépülő programokat vagy további könyvtárakat újra linkelnénk.

  – Tehát, noha a dinamikusan linkelhető függvénykönyvtárak biztosítanak egyfajta rugalmasságot, használatuk mégis maga után von némi komplikációt. Alkalmazá-

[12] Az `-fPIC` kapcsolót olyankor is megadjuk, amikor arra számítunk, hogy a statikusan linkelhető könyvtárat dinamikusan linkelhető könyvtár részeként szeretnénk felhasználni.

suk esetén a program betöltésekor a linkelés egy fázisa hátramarad a futtatásig: a program indításakor az operációs rendszernek meg kell találnia a meghivatkozott, lefordított megosztott objektum fájlokat. GNU/Linux platformon, ELF formátumú futtatható állományok esetén az 5.2. táblázatban foglalt futásidejű iránymutatásokat tudjuk az operációs rendszernek átadni az utóbbi folyamatot segítendő, ami azonban ezeket nem köteles figyelembe venni.

Mikor használható egyik, illetve a másik könyvtártípus

A projekt, az általában idegen (a fejlesztői rendszertől különböző) architektúrájú kulcstároló eszköz oldalon felhasznált legtöbb - de nem szükségszerűen mindegyik - könyvtára, szerveződjenek azok GNU Autotools, CMake, vagy egyéb rendszerben, fordítás előtt lehetőséget kínál a kettő közül bármelyik formátumú könyvtár állomány előállítására (6.2. ábra: „Primer" függőségek). Ahol mégsem lenne elérhető mindkét opció, ott eddigi tapasztalataink szerint általában csak a dinamikusan linkelhető könyvtár állomány(ok) generálását lehet választani.

Az Android készülékkel megvalósítandó céljaink

A 6.3. pontban, első lépésben csupán arra fogunk kísérletet tenni, hogy egy harmadik féltől származó könyvtárakat felhasználó állományt futtassunk az Android operációs rendszer héjparancsértelmezőjében (6.2. ábra: felső sor). A későbbiekben (6.4. pont) azonban a végső célunk az, hogy a projekt felhasználója elől a technikai részleteket elrejtve, egy Android alkalmazás felületén keresztül tegyük lehetővé a kulcstároló implementáció használatára (6.2. ábra: alsó sor).

A választott irányelv

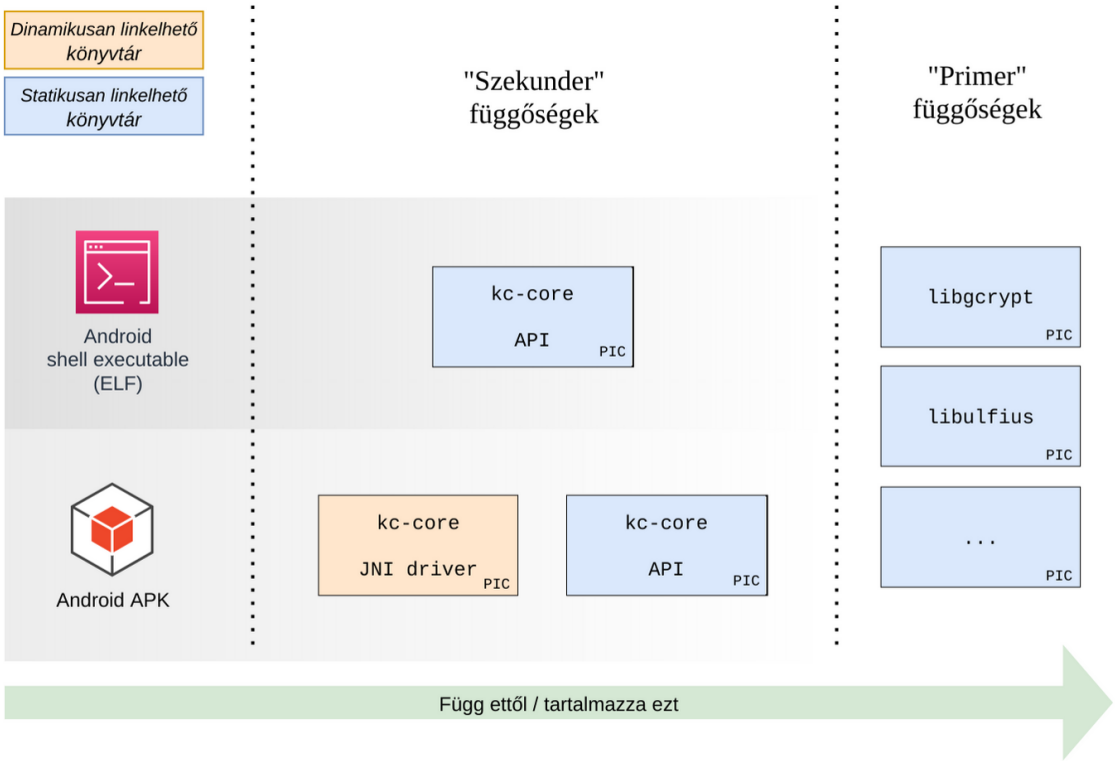
Az ebben az alpontban megfogalmazott, elsőre egyszerűnek tűnő kérdésnek a kimenetelét, amint láthatjuk, számos tényező befolyásolja. A fentieket egybevetve, az alábbi döntéseket hoztuk meg:

- Közvetlen, héjból történő futtatás esetében (6.2. ábra: felső sor) a legegyszerűbben vélhetőleg akkor járhatunk, ha statikusan linkelhető könyvtárakat használunk fel a tárgykód generálása során. Ekkor az utóbbi optimális méretű lesz, és nem fog problémát okozni az a kérdés, hogy az Android operációs rendszer hogyan találja meg a meghivatkozott megosztott függvénykönyvtárakat (tapasztalatunk alapján, héjból történő futtatás esetén ez nehezen oldható meg).

  - Az Android oldalon igényelt 6.2. ábra szerinti „Primer” függőségeket, amennyiben az lehetséges, mindig statikusan linkelhető változatban fogjuk a célplatformra lefordítani. Az írás pillanatában ez mindegyik „Primer” könyvtárra igaz volt.

- Android alkalmazásból történő futtatáskor (6.2. ábra: alsó sor) JNI felületen keresztül történő hívás esetén már van lehetőségünk a dinamikusan linkelt könyvtárak manuális, egyesével történő betöltésére a programban. Jelenlegi tudomásunk szerint a JNI eleinte dinamikusan linkelhető könyvtárak betöltését támogatta csak, ezért az alkalmazáson keresztül ilyen könyvtárat fogunk használni.

  - A JNI illesztésért felelős dinamikusan linkelhető könyvtárba (lásd a 6.4. pontot, és a 6.2. ábrán a „JNI driver” modult) a statikusan linkelhető `kc-core` API könyvtárat, illetve a „Primer” függőségeket linkeljük, létrehozva így egyetlen új modult, ami közvetlenül felhasználható az alkalmazásból.



6.2. ábra. A teszt parancssori program és az Android alkalmazás linkelése

6.3. Egy példa program futtatása az Android parancsértelmezőben

Az Android operációs rendszerre szánt, parancssorban futtatható állomány (6.2. ábra: felső sor) forráskódja az F.7. listázásban olvasható. Láthatóan ebben a teszt program nem tesz egyebet, minthogy elindítja a szervert (egy nem blokkoló függvény hívásával), és fenntartja a program futását.

A `kc_start_accepting_requests()` implementációja az 5777-es TCP porton elindít egy HTTP szervert, mely a `/hello` erőforrás alatt visszaküld egy statikus JSON választ.

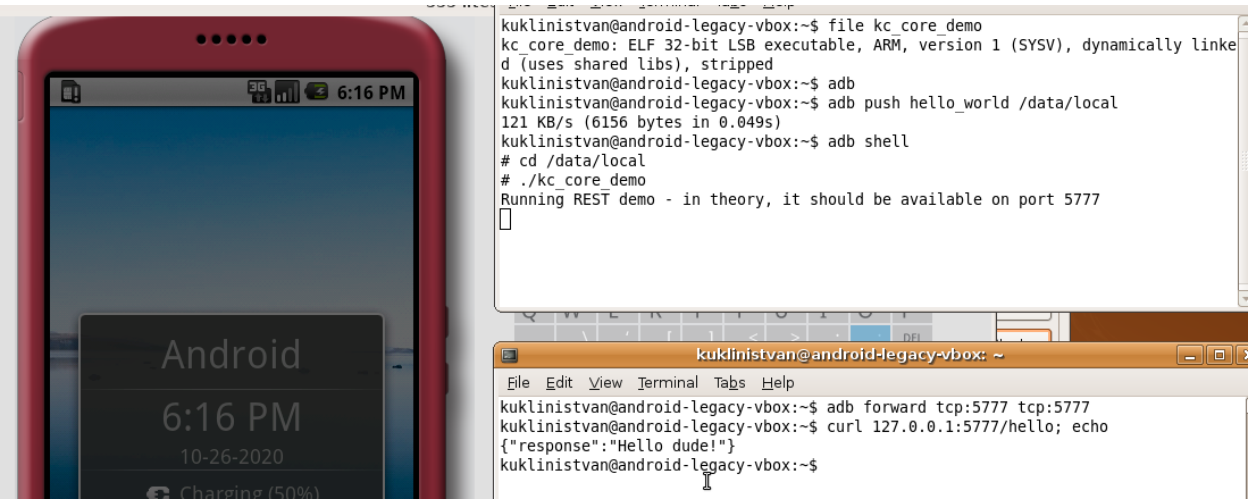
A szükséges fordítói argumentumok egy része a K.21. listázásban olvasható - látható többek között a célarchitektúrának megfelelő fordító meghívása, a célarchitektúra kijelölése, a legmagasabb szintű optimalizáció kiválasztása és az `NDEBUG` („not debug") makró beállítása. Az 5.4.4. pontban ismertetett könyvtárütközésekkel kapcsolatos probléma orvoslásának automatizálására a Conan[13] csomagkezelő programot hívtuk segítségül, ami nagyban megkönnyítette a könyvtárak platformonként lefordított tárgykódjainak elkülönítését, és a fordítási folyamat megismételhetőségének megszervezését. A példa program tárgykódjának generálásakor futtatott parancsállományunk kimenetének részlete a K.22. listázásban olvasható, amint beállítja, elsősorban a könyvtárakra vonatkozó környezeti változókat futtatható állomány előállítása előtt.

Sikeres fordítást követően a létrejött `kc_core_demo` futtatható állományt feltöltöttük a virtuális készülékre, és elindítottuk a programot. Azért, hogy az előbbi által szolgáltatott REST felülethez hozzáférjünk az emulációt megvalósító számítógépen, az ADB segítségével átirányítottuk a korábban említett portot a visszacsatolt hálózati interfészre (a `127.0.0.1` címre). A `curl` program segítségével meggyőződtünk arról, hogy az általunk előkészített végpont a várt választ adja. (6.3. ábra)

Nagy örömünkre, megállapíthattuk, hogy nem csak elméletileg lehetséges - legalább néhány esetben - nagyobb, összetettebb, akár asztali környezetből ismert könyvtárakat az Android operációs rendszerrel ellátott eszközön felhasználni.

A dolgozat utolsó részében bemutatjuk, hogyan sikerült az ugyanezen könyvtárunk által rendelkezésre bocsátott függvényeket Android alkalmazáson keresztül hívni.

[13] `https://conan.io/`



6.3. ábra. A `kc_core_demo` futtatása a virtuális, korai Android eszközön

6.4. Az elkészült könyvtár használata az alkalmazásban JNI segítségével

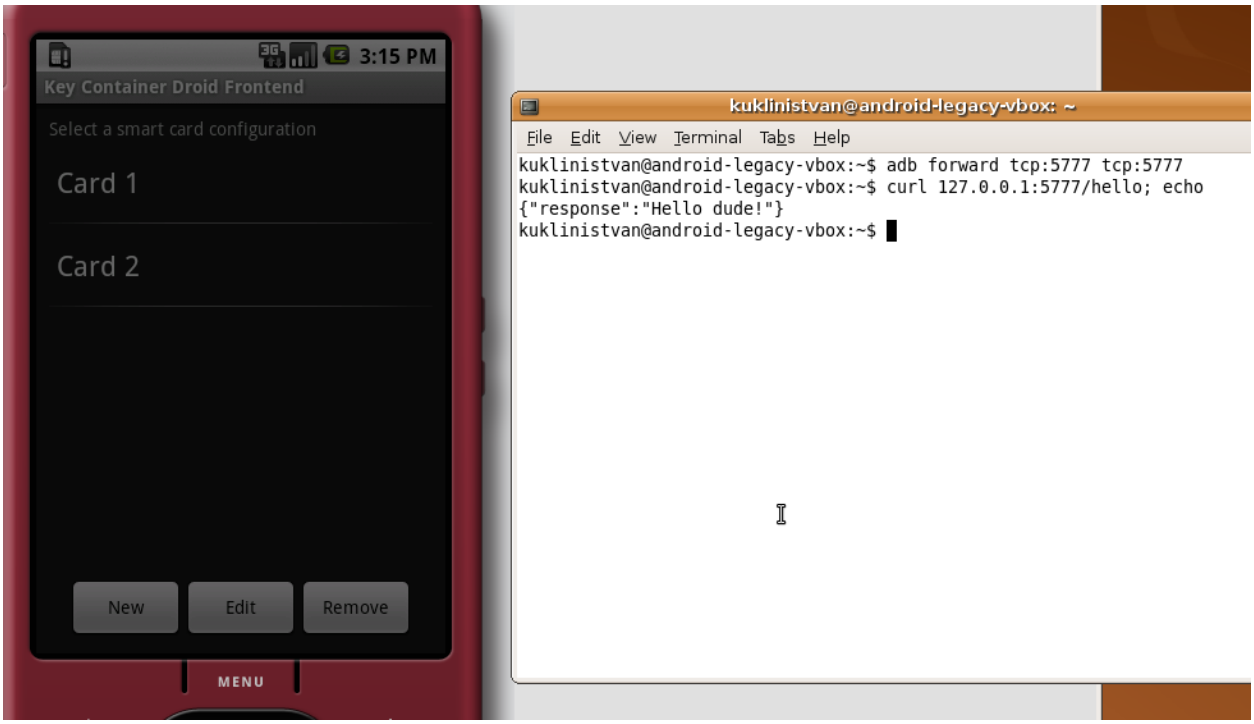
Az A. mellékletben bemutatott fejlesztői környezetben létrehoztunk egy alkalmazást, és annak a betöltő rutinjában (az ún. activity `onCreate()` függvényében) elhelyeztük a hívást, ami elindítja a 6.1.1. pontban tárgyalt REST kiszolgálót (F.10. listázás).

Ennek pontosabb megvalósításához tekintsük a kezdetleges `kc-core` API felületet - az elérni kívánt függvényeket - az F.13. listázásban.

A Java platformhoz való illesztéshez első lépésben létrehoztuk az API által szolgáltatott függvények fejléceit tartalmazó osztályt, az F.6. listázásban látható módon. Ebből a Sun Java JDK 6 programcsomag `javah` eszközének segítségével előállítható az F.11. listázásban olvasható, C nyelvbeli fejlécállomány, amit `kc-core` API hívások segítségével implementáltunk az F.12. listázásban olvasható módon. Megjegyezzük, hogy hibakeresés céljából segítségünkre vált, hogy első lépésben egyszerű kiíratásokat helyeztünk csak a függvénytörzsekbe.

Végül - ismételten, segítségül hívva a Conan csomagkezelőt - utolsó dolgunk volt, hogy a 6.2. ábrán `kc-core` API modult és a „Primer” függőségeket a programhoz linkeljük.

A keletkezett függvénykönyvtárat a projektkönyvtáron belül a megfelelő architektúra natív függőségei között helyeztük el, a `libs/armeabi` útvonalon. A projekt ismételt építése után előállt az Android telefonra telepíthető alkalmazást csomagoló `.apk` állomány, mely tartalmazza az iménti natív modult.



6.4. ábra. Az alkalmazáson keresztül futtatott natív modul sikeresen válaszol a REST kérelmekre

Noha ez korántsem sikerült az első próbálkozásra, de kitartó hibakereséssel, és a komponensek többszöri újrafordításával töltött munkamenetek után, az eredmény a 6.4. ábrán látható.

7. Összefoglalás

Elért célok, eredmények

Noha ezen dolgozat keretében még a 3.2.1. pontban kijelölt célközönség számára használható termék nem állt elő, azonban azt mindenképpen megállapítjuk, hogy a 3. pontban kitűzött cél minden valószínűséggel elérhető. A nyílt forrású GnuPG programban használatos kulcsok védelmére felhasználható intelligens kártya helyettesítésére a 3.3.4. pont szempontjainak megfeleltethető egyéni eszköz, ami így nagyobb biztonság szavatolása mellett, az emulációnak köszönhetően, az ergonomikus használat lehetőségét is kínálja.

Az 5.3. pontban áttekintettük a GnuPG ökoszisztéma intelligens kártya támogatásának az architektúráját. A lehetséges illesztési pontok közül az `scdaemon` programot sikeresen egészítettük ki úgy, hogy utóbbi egyéni - nem feltétlenül USB kapcsolaton keresztül megközelíthető, nem feltétlenül az OpenPGP Smart Card szoftvert futtató - kártyát kezelő implementáció betöltésére alkalmas legyen (5.4.3. pont). Egy ilyen egyéni kártyaimplementációval elértük, hogy a GnuPG felismerjen egy fiktív kártyaolvasót, kártyát, és azon egy fiktív kulcsot (K.19. és K.23. listázás). Ezt megvalósítandó sikeresen kiolvastuk egy GnuPG RSA kulcsból annak nyilvános paramétereit (K.20. listázás), amit fel tudtunk küldeni az `scdaemon` komponensből a GnuPG komponensei felé (5.6.3. pont).

Ezen törekvésünkkel párhuzamosan módot találtunk arra, hogy egy kis teljesítményű, (a fejlesztői rendszer viszonylatában) idegen processzor architektúrájú, Linux kernelt futtatni képes eszközön, a fejlesztői rendszeren is elérhető könyvtárakat használjunk fel. A virtuális Android készüléken le tudtuk futtatni a teszt parancssori alkalmazást (6.3. pont), és azt követően az Android alkalmazást is (6.4. pont) - majd a végpontról képesek voltunk az általuk kiszolgált REST felülettel kommunikálni.

Továbbfejlesztés

Az 5.4.2. pontban szereplő parancsok közül idáig a `LEARN`, `SERIALNO` és a `READKEY` parancsokat implementáltuk, így elérve, hogy a korábban említett fiktív objektumok megjelenjenek a GnuPG rendszerben. Ezek tehát kriptográfiai műveleteket valójában még nem látnak el.

Ez még sem aláírásra, sem titkosításra nem alkalmas: ehhez további `scdaemon` utasítások megvalósítása szükséges.

- A 6.1. pontban ismertetett statikus implementációhoz vélhetőleg elegendőek lesznek

- legalább is az esetek többségében - előre bekészített adatokat visszaküldeni, amik remélhetőleg alkalmasak lesznek egyetlen egy fájl egyféle módon történő aláírására.

- Teljes implementáció esetén az ehhez szükséges könyvtárakat (elsősorban: `libgcrypt`) is le kell fordítani Android operációs rendszerre. A hiányzó könyvtár(ak) fordítása az eddigi tapasztalatok alapján valószínűleg sikeres lesz, de lehet, hogy - mint ahogyan számos esetben a 6.1. táblázatban felsorolt könyvtárak esetén is - szükség lesz további finomhangolásokra, esetleg foltozás készítésére.

  – Itt megjegyezzük, hogy az kétségkívül a könnyíti az előrehaladást, hogy egy intelligens kártyával ellentétben itt fel tudjuk használni a GnuPG ökoszisztéma saját könyvtárait (5.1. táblázat), ezáltal könnyebben betartva az `scdaemon` által definiált protokollt.

- Természetesen az implementáció nem csak RSA [24], hanem ECC [14] [19] és DSA [21] kulcsok támogatására is kiterjedhet.

Úgy érezzük, a projektbe fektetett energiánk megtérült, hiszen, mialatt az utóbbival foglalkoztunk, sok új tapasztalatra tettünk szert az alábbi területeken:

- nyílt forráskódú alkalmazások szerveződése,
- alacsony szintű szoftverfejlesztés:
  – statikus és dinamikus könyvtárak szerveződése,
  – szoftverépítő és könyvtárkezelő (csomagkezelő) rendszerek,
  – idegen processzor architektúrás platform támogatása,
  – hibakeresés, fejlesztői munkamenet.
- fejlesztés már nem támogatott, korai eszközökre,
- GNU/Linux ökoszisztéma.

Ezúton is köszönjük az Olvasó idejét, amit a dolgozatunk tanulmányozására szánt! A továbbiakban a mellékletek következnek.

8. Mellékletek

A. Alkalmazásfejlesztő környezet korai Android eszközökhöz

A 6.2.4. pontban ismertetett natív könyvtárakkal némi előkészület után viszonylag zökkenőmentes volt az előrehaladás - ideértve azt is, hogy a 6.2. pontban az Android operációs rendszerrel kompatibilis, futtatható ELF állomány előállt.

A korai Android 1.5 (Cupcake) operációs rendszert támogató emulátor, és az Eclipse integrált fejlesztői környezet üzembe helyezése ugyanakkor már nem volt teljesen zökkenőmentes, mivel ezek a platformverziók egy ideje nem támogatottak.

Internetes archívumok segítségével, a kompatibilis fejlesztői környezet kiépítéséhez felderítettük a szoftverfüggőségeknek - többek között az operációs rendszernek - az időben egymáshoz közel eső kiadásait, verziót, így ebből össze tudtunk állítani egy fejlesztői virtuális gépet. Utóbbi látható működés közben többek között a 6.3. ábrán, és ezt használjuk a K.25. listázásban is.

Az A.1. táblázatban azok a szoftververziók olvashatóak, amikkel sikeresen tudtunk dolgozni.

Szoftverkomponens	Verzió	Kiadás ideje
Android platform	1.5	2009. április
Ubuntu Linux	8.04 LTS (i368 arch.)	2008. április
VirtualBox és integrációs szolgáltatások	6.1.4	2020. február
Android SDK	1.6 r1	2009. szeptember
Sun Java JDK	6	2006. december
Eclipse	Ganymede, 3.4.2	2008. június
Eclipse ADT	0.9.5	2009. december
Android NDK	r10e	2015. május

A.1. táblázat. Az Android 1.5 operációs rendszert támogató fejlesztőkörnyezet felépítése

Megjegyezzük, hogy ez a környezet csak az alkalmazásfejlesztéshez, és azok kipróbálásához volt szükséges. Az Android platformon, ARM processzoros architektúrán futtatható gépi kód 6.2. pontban tárgyalt előállításához a modern fejlesztői operációs rendszerünk is megbízhatóan használhatónak bizonyult.

F. Forráskód részletek gyűjteménye

```
static int
register_commands (assuan_context_t ctx)
{
    static struct {
    const char *name;
    assuan_handler_t handler;
    const char * const help;
    } table[] = {
    { "SERIALNO",     cmd_serialno, hlp_serialno },
    { "LEARN",        cmd_learn,    hlp_learn },
    // ...
    { "DISCONNECT",   cmd_disconnect,hlp_disconnect },
    { "APDU",         cmd_apdu,     hlp_apdu },
    { NULL }
    };
    // ...
    return 0;
}
```

F.1. listázás. `scdaemon`: az Assuan parancsok regisztrálása

```
/* Implement this file in a shared library.
 * Load with the --custom-backend option. */

#include <assuan.h>
#include <gpg-error.h>

gpg_error_t custom_cmd_serialno (assuan_context_t ctx, char *line);
gpg_error_t custom_cmd_learn (assuan_context_t ctx, char *line);
gpg_error_t custom_cmd_readcert (assuan_context_t ctx, char *line);
// ...
```

F.2. listázás. `scdaemon`: egyénileg implementálható függvények

```c
static gpg_error_t
cmd_learn (assuan_context_t ctx, char *line)
{
    ctrl_t ctrl = assuan_get_pointer (ctx);
    int rc = 0;
    // ...
    if (!only_keypairinfo)
    {
        // ...
        reader = apdu_get_reader_name (app->slot);
        // ...
        if (!serial)
            return gpg_error (GPG_ERR_INV_VALUE);

        rc = assuan_write_status (ctx, "SERIALNO", serial);
        // ...
    }
    // ...
    return rc;
}
```

F.3. listázás. `scdaemon`: részlet a `LEARN` parancs implementációjából

```c
#include <kc/scd/backend-custom.h>
#include "../../debug.h"
///...
#define KC_LOG_THIS_CMD() {\
///...
gpg_error_t custom_cmd_serialno (assuan_context_t ctx, char *line) {
    KC_LOG_THIS_CMD();
    return 0;
}

gpg_error_t custom_cmd_learn (assuan_context_t ctx, char *line) {
    KC_LOG_THIS_CMD();
    return 0;
}
///...
```

F.4. listázás. `scdaemon`: „noop” kártyamodul implementáció.

```c
gpg_error_t custom_cmd_learn (assuan_context_t ctx, char *line) {
    KC_LOG_THIS_CMD();

    if(assuan_write_status(ctx, "READER", DUMMY_READER_NAME)) return 1;
    if(assuan_write_status(ctx, "SERIALNO", DUMMY_SERIALNO)) return 1;
    if(assuan_write_status(ctx, "APPTYPE", "openpgp")) return 1;
    if(assuan_write_status(ctx, "DISP-NAME", "John Doe")) return 1;

    log("cmd_learn done");
    return 0;
}
```

F.5. listázás. `scdaemon`: a minimális `SERIALNO` paraméteren felül néhány egyéb felküldése a gpg kliensnek

```java
package com.codekuklin.kc_droid_frontend;

public class KC_Core_JNI {
    static {
        System.loadLibrary("kc-core-bridge");
    }

    public native void startAcceptingConnections();
    public native void stopAcceptingConnections();
}
```

F.6. listázás. A C nyelven írt, natív függvényekre hivatkozó JNI illesztő osztály

```c
#include <kc.h>
#include <unistd.h>
#include <stdio.h>

int main() {
    printf("Running REST demo - in theory, it should be available on port 5777\n");
    kc_start_accepting_requests();
    while (1) sleep(10);
    return 0;
}
```

F.7. listázás. Példa a kezdetleges `kc-core` indítására az API felületen keresztül

```c
gpg_error_t custom_cmd_learn (assuan_context_t ctx, char *line) {
    if(write_johndoes_fingerprints(ctx, log)) return 1;
    // (...)
    snprintf(buf, buf_len, "%s %s", johndoe_rsa_asc_keygrip,
    ↪  johndoe_rsa_asc_fingeprint);
    log("Reporting John Doe's public key");
    if(assuan_write_status(ctx, "KEYPAIRINFO", buf)) return 1;
    // (...)
}

static int write_johndoes_fingerprints(assuan_context_t ctx, log_fn_t log) {
    /// (...)
    log("Sending KEY-FPR: signature (1)");
    sprintf(fpr_response_buf, "1 %s", johndoe_rsa_asc_fingeprint);
    if(assuan_write_status(ctx, "KEY-FPR", fpr_response_buf)) return 1;
    // (...): Same for encryption & authentication slots.
}
```

F.8. listázás. A nyilvános kulcsok lenyomatainak felküldése

```c
gpg_error_t custom_cmd_readkey (assuan_context_t ctx, char *line) {
    // (...)
    if(!strcmp(line, johndoe_rsa_asc_fingeprint) || !(strcmp(line,
    ↪  johndoe_rsa_asc_keygrip))) {
        if(sendback_johndoe_public_sexp(ctx)) return 1;
    }
    // (...)
}
// (...)
static int sendback_johndoe_public_sexp(assuan_context_t ctx) {
    log("cmd_readkey - Printing csexp");
    if(!gcry_sexp_sprint(rsa_public_sexp, GCRYSEXP_FMT_CANON, canon_sexp_buffer,
    ↪  canon_sexp_length)) return 1;
    log("cmd_readkey - Sending csexp using assuan_send_data");
    if(assuan_send_data(ctx, canon_sexp_buffer, canon_sexp_length)) return 1;
}
```

F.9. listázás. A nyilvános RSA paraméterek kommunikálása csexp formátumban

```java
package com.codekuklin.kc_droid_frontend;
// (...)
public class SmartCardConfigurations extends Activity {
    @Override
    public void onCreate(Bundle savedInstanceState) {
        // (...)
        KC_Core_JNI kc = new KC_Core_JNI();
        kc.startAcceptingConnections();
        // (...)
    }
}
```

F.10. listázás. A `kc-core` kiszolgáló indítása az alkalmazás betöltésekor (lásd még: F.11. listázás)

```cpp
/* DO NOT EDIT THIS FILE - it is machine generated */
#include <jni.h>
/* Header for class com_codekuklin_kc_droid_frontend_KC_Core_JNI */

#ifndef _Included_com_codekuklin_kc_droid_frontend_KC_Core_JNI
#define _Included_com_codekuklin_kc_droid_frontend_KC_Core_JNI
#ifdef __cplusplus
extern "C" {
#endif
/*
 * Class:     com_codekuklin_kc_droid_frontend_KC_Core_JNI
 * Method:    startAcceptingConnections
 * Signature: ()V
 */
JNIEXPORT void JNICALL
↪  Java_com_codekuklin_kc_1droid_1frontend_KC_1Core_1JNI_startAcceptingConnections
  (JNIEnv *, jobject);

// (...)

#ifdef __cplusplus
}
#endif
#endif
```

F.11. listázás. Az F.10. listázásban olvasható osztályból generált natív fejlécállomány (lásd még a kézileg megírt implementációját az F.12. listázásban)

```
#include <stdio.h>
#include <com_codekuklin_kc_droid_frontend_KC_Core_JNI.h>
#include <kc/kc.h>

JNIEXPORT void JNICALL
↪  Java_com_codekuklin_kc_1droid_1frontend_KC_1Core_1JNI_startAcceptingConnections
  (JNIEnv * env, jobject obj)
{
    kc_start_accepting_requests();
}

JNIEXPORT void JNICALL
↪  Java_com_codekuklin_kc_1droid_1frontend_KC_1Core_1JNI_stopAcceptingConnections
  (JNIEnv * env, jobject obj)
{
    kc_stop_accepting_requests();
}
```

F.12. listázás. Az F.11. listázásban szereplő fejlécállomány implementációja, az F.13. listázásban olvasható `kc-core` API függvényeivel

```
#pragma once
/// Non-blocking
void kc_start_accepting_requests();
/// Non-blocking
void kc_stop_accepting_requests();
```

F.13. listázás. A kezdetleges, natív `kc-core` API

K. Parancssori kimenetek gyűjteménye

```
$ ./scdaemon --server
scdaemon[632506]: NOTE: this is a development version!
OK GNU Privacy Guard's Smartcard server ready
HELP
(...)
# PKSIGN [--hash=[rmd160|sha{1,224,256,384,512}|md5]] <hexified_id>
# PKAUTH <hexified_id>
# PKDECRYPT <hexified_id>
(...)
# GETATTR <name>
# SETATTR <name> <value>
# WRITECERT <hexified_certid>
# WRITEKEY [--force] <keyid>
# GENKEY [--force] [--timestamp=<isodate>] <no>
# RANDOM <nbytes>
# PASSWD [--reset] [--nullpin] [--clear] <chvno>
# CHECKPIN <idstr>
(...)
# APDU [--[dump-]atr] [--more] [--exlen[=N]] [hexstring]
# KILLSCD
OK
```

K.14. listázás. Az `scdaemon` által felkínált lehetőségek egy része

```
HELP RANDOM
# RANDOM <nbytes>
#
# Get NBYTES of random from the card and send them back as data.
# This usually involves EEPROM write on the card and thus excessive
# use of this command may destroy the card.
#
# Note, that this function may be even be used on a locked card.
OK
```

K.15. listázás. Az `scdaemon` súgószöveget biztosít minden elérhető parancshoz

```
$ scdaemon --help
scdaemon (GnuPG) 2.2.20-beta16
(...)
Syntax: scdaemon [options] [command [args]]
Smartcard daemon for GnuPG

Options:
     --server                 run in server mode (foreground)
(...)
     --custom-backend FILE    use custom smart card backend

(...)
```

K.16. listázás. `scdaemon`: új opció egyéni kártyaimplementációval történő indításra

```
$ gpg --debug-level guru --card-status
gpg: enabled debug flags: packet mpi crypto filter iobuf memory cache memstat trust
↪  hashing ipc clock lookup extprog
gpg: DBG: [not enabled in the source] start
gpg: DBG: chan_3 <- OK Pleased to meet you, process 475924
(...)
gpg: DBG: chan_3 -> SCD GETINFO version
gpg: DBG: chan_3 <- D 2.2.20-beta16
gpg: DBG: chan_3 <- OK
gpg: WARNING: server 'scdaemon' is older than us (2.2.20-beta16 < 2.2.23)
gpg: Note: Outdated servers may lack important security fixes.
gpg: Note: Use the command "gpgconf --kill all" to restart them.
gpg: DBG: chan_3 -> SCD SERIALNO
(...)
```

K.17. listázás. A GnuPG felfedi a megfelelő kapcsolóval a kommunikációját a többi modul irányába

```
$ scdaemon --server
scdaemon[472507]: Custom smart card backend requested: /home/kuklinistvan/Roaming
↪  Work/Key_Container_proj
/kc-emulator/lib/libscd_custom_dummy.so
scdaemon[472507]: NOTE: this is a development version!
OK GNU Privacy Guard's Smartcard server ready
learn
S READER Dummy reader
S SERIALNO D276000124010340FFFF123456780000
S APPTYPE openpgp
S DISP-NAME John Doe
S KEY-FPR 1 08FFBAB3FE6A3CF53ECD90188E242886C1A59F79
S KEY-FPR 2 08FFBAB3FE6A3CF53ECD90188E242886C1A59F79
S KEY-FPR 3 08FFBAB3FE6A3CF53ECD90188E242886C1A59F79
S KEYPAIRINFO 665FF8AED2C1F8E775B35DF5094B596A3B2919E4
↪  08FFBAB3FE6A3CF53ECD90188E242886C1A59F79
(...)
```

K.18. listázás. `scdaemon`: a `LEARN` parancsra adott válaszok (részlet)

```
Reader ...........: Dummy reader
Application ID ...: D276000124010340FFFF123456780000
Application type .: OpenPGP
Version ..........: 3.40
Manufacturer .....: ?
Serial number ....: 12345678
Name of cardholder: John Doe
Language prefs ...: [not set]
Salutation .......:
URL of public key : [not set]
Login data .......: [not set]
Signature PIN ....: forced
Max. PIN lengths .: 0 0 0
PIN retry counter : 0 0 0
Signature counter : 0
Signature key ....: [none]
Encryption key....: [none]
Authentication key: [none]
General key info..: [none]
```

K.19. listázás. A `gpg` kliens sikeresen észleli az üres virtuális kártyát.

```
-- Finding secret key packet
-- Secret key packet found at at 0x0
-- Detected packet version 4
-- Key algorithm is RSA as expected.
-- Reading public RSA parameters
-- Reading MPI
-- MPI length: 2048 bits according to header
-- Read (258 bytes = 2064 bits) = (256 bytes = 2048 bits body) + (2 bytes = 16 bits
↪  header)

-- Reading MPI
-- MPI length: 17 bits according to header
-- Read (5 bytes = 40 bits) = (3 bytes = 24 bits body) + (2 bytes = 16 bits header)

-- Reading private RSA parameters
-- string to key octet 0, ok
```

K.20. listázás. Részlet az RSA kulcsolvasó modul naplójából

```
/opt/kuklinistvan/android-ndk-r10e/STANDALONE/arm/bin/arm-linux-androideabi-gcc \
    --sysroot=/opt/kuklinistvan/android-ndk-r10e/STANDALONE/arm/sysroot \
    -D_GLIBCXX_USE_CXX11_ABI=0  \
    (...)   \
    -s  \
    \
    -march=armv5te  \
    -mthumb \
    -msoft-float    \
    (...)
    \
    -pthread    \
    -O3 \
    -DNDEBUG    \
    \
    -o CMakeFiles/kc_core_demo.dir/kc_core_demo.c.o \
    -c "/home/kuklinistvan-l/Roaming
    ↪  Work/Key_Container_proj/kc-core/demos/libraries_linked/kc_core_demo.c"
```

K.21. listázás. A kezdetleges `kc-core` API-val írt program fordítása CMake segítségével

```
Setting up environment:
{ 'CFLAGS': '-O3 -s',
  'CXXFLAGS': '-O3 -s '
              '-I/home/kuklinistvan-l/.conan/data/ulfius/2.6.5/_/_/package/(...)/include
              ↪  '
              (...)
              '-DNDEBUG -D_GLIBCXX_USE_CXX11_ABI=0',
  'LDFLAGS': '-L/home/kuklinistvan-l/.conan/data/ulfius/2.6.5/_/_/package/(...)/lib '
             (...)
  'LIBS': '-lulfius -lorcania -lyder -lgnutls -lmicrohttpd -ljansson -lcurl '
          '-lidn2 -lnettle -lhogweed -ltasn1 -lunistring -lgmp'}
```

K.22. listázás. A környezeti változók beállítása a kezdetleges `kc-core` API-val írt demó program fordítása előtt

```
Signature key ....: 08FF BAB3 FE6A 3CF5 3ECD  9018 8E24 2886 C1A5 9F79
Encryption key....: 08FF BAB3 FE6A 3CF5 3ECD  9018 8E24 2886 C1A5 9F79
Authentication key: 08FF BAB3 FE6A 3CF5 3ECD  9018 8E24 2886 C1A5 9F79
General key info..: pub  rsa2048/8E242886C1A59F79 2020-09-08 John Doe <john@doe>
sec#  rsa2048/8E242886C1A59F79  created: 2020-09-08  expires: soha
ssb>  rsa2048/B28E6C292ADF668B  created: 2020-09-08  expires: soha
                                card-no: FFFF 12345678
```

K.23. listázás. A GnuPG sikeresen kiolvasta a nyilvános RSA paramétereket

```
# cat /proc/cpuinfo
Processor        : ARM926EJ-S rev 5 (v5l)
(...)
Features        : swp half thumb fastmult vfp edsp java
CPU implementer        : 0x41
CPU architecture: 5TEJ
CPU variant        : 0x0
CPU part        : 0x926
CPU revision        : 5
(...)
```

K.24. listázás. Az emulált Android 1.5 telefon ARM processzora (`adb shell` parancssori kimenet)

```
$ arm-linux-androideabi-gcc hello_world.c -o hello_world
$ file hello_world
hello_world: ELF 32-bit LSB executable, ARM, EABI5 version 1 (SYSV), dynamically linked,
↪  interpreter /system/bin/linker, not stripped
$ scp hello_world kuklinistvan@192.168.56.101:~
hello_world                                   100% 6156     2.2MB/s   00:00
$ ssh kuklinistvan@192.168.56.101
Welcome back to the Android developer VM!
Last login: Fri Oct 23 11:14:46 2020 from 192.168.56.1
kuklinistvan@android-legacy-vbox:~$ adb push hello_world /data/local
* daemon not running. starting it now *
* daemon started successfully *
117 KB/s (6156 bytes in 0.051s)
kuklinistvan@android-legacy-vbox:~$ adb shell
# /data/local/hello_world
Hello world!
#
```

K.25. listázás. Hello World program fordítása és futtatása Android eszközön

Ábrák jegyzéke

1.1. A böngésző jelzi a titkosítást 6

1.2. Az információ sebezhetősége útközben 7

2.1. Nyílt kulcsú titkosítás és aláírás vázlatosan 15

3.1. Vállalatot azonosító EV SSL tanúsítvány 17

3.2. Intelligens kártya 22

4.1. A kulcstároló eszköz kapcsolatai a külvilággal és a gazda hardverrel - Android eszköz esetén 27

4.2. A kulcstároló eszköz gyakorlati tekintetben éppen egy intelligens kártya szerepét tölti be 28

5.1. A tényleges és az okostelefon segítségével emulált intelligens kártya kommunikációs illesztései 30

5.2. GnuPG intelligens kártya architektúra [11] 31

6.1. Statikusan és dinamikusan hivatkozható (linkelhető) könyvtárak fordítása . 49

6.2. A teszt parancssori program és az Android alkalmazás linkelése 52

6.3. A `kc_core_demo` futtatása a virtuális, korai Android eszközön 54

6.4. Az alkalmazáson keresztül futtatott natív modul sikeresen válaszol a REST kérelmekre 55

Listázások jegyzéke

5.1. A GnuPG komponenseinek fordítása 34

5.2. Publikus RSA paraméterek a `libgcrypt` sexp formátumában 43

6.1. GNU Autotools projekt konfigurálása Android rendszerre történő fordításra 47

6.2. CMake projekt konfigurálása Android rendszerre történő fordításra 47

F.1. `scdaemon`: az Assuan parancsok regisztrálása 59

F.2. `scdaemon`: egyénileg implementálható függvények 59

F.3. `scdaemon`: részlet a `LEARN` parancs implementációjából 60

F.4. `scdaemon`: „noop” kártyamodul implementáció. 60

F.5. `scdaemon`: a minimális `SERIALNO` paraméteren felül néhány egyéb felküldése a `gpg` kliensnek 61

F.6. A C nyelven írt, natív függvényekre hivatkozó JNI illesztő osztály 61

F.7. Példa a kezdetleges `kc-core` indítására az API felületen keresztül 61

F.8. A nyilvános kulcsok lenyomatainak felküldése 62

F.9. A nyilvános RSA paraméterek kommunikálása csexp formátumban 62

F.10. A `kc-core` kiszolgáló indítása az alkalmazás betöltésekor (lásd még: F.11. listázás) 63

F.11. Az F.10. listázásban olvasható osztályból generált natív fejlécállomány (lásd még a kézileg megírt implementációját az F.12. listázásban) 63

F.12. Az F.11. listázásban szereplő fejlécállomány implementációja, az F.13. listázásban olvasható `kc-core` API függvényeivel 64

F.13. A kezdetleges, natív `kc-core` API 64

K.14. Az `scdaemon` által felkínált lehetőségek egy része 65

K.15. Az `scdaemon` súgószöveget biztosít minden elérhető parancshoz 65

K.16. `scdaemon`: új opció egyéni kártyaimplementációval történő indításra 66

K.17. A GnuPG felfedi a megfelelő kapcsolóval a kommunikációját a többi modul irányába 66

K.18. `scdaemon`: a `LEARN` parancsra adott válaszok (részlet) 67

K.19. A `gpg` kliens sikeresen észleli az üres virtuális kártyát. 67

K.20. Részlet az RSA kulcsolvasó modul naplójából 68

K.21. A kezdetleges `kc-core` API-val írt program fordítása CMake segítségével . . 68

K.22. A környezeti változók beállítása a kezdetleges `kc-core` API-val írt demó program fordítása előtt 69

K.23. A GnuPG sikeresen kiolvasta a nyilvános RSA paramétereket 69

K.24. Az emulált Android 1.5 telefon ARM processzora (`adb shell` parancssori kimenet) 69

K.25. Hello World program fordítása és futtatása Android eszközön 70

Táblázatok jegyzéke

4.1. Példák a kulcstároló eszköz (`kc_core`) megvalósítására többféle eszközön . . 28

5.1. Az általunk használt GnuPG komponensek verziói 34

5.2. A könyvtárak meghivatkozása fordítási és futási időben, GNU/Linux platformon 38

5.3. A statikus kártyamodul AID azonosítójának felépítése 40

5.4. Részlet a secret-key packet-ből kiolvasható információkból, a próba kulcsra vetítve 41

6.1. A `kc-core` komponensben megvalósított REST felület lefordításához szükséges minden köztes, általunk felhasznált könyvtárfüggőség 48

A.1. Az Android 1.5 operációs rendszert támogató fejlesztőkörnyezet felépítése . . 58

Szójegyzék

Android Debug Bridge Android operációs rendszert futtató telefonhoz alkalmazásfejlesztő eszköz. Egy alkalmazás hibakereső módban történő futtatásakor a számítógéphez csatlakoztatott telefonon futó programot a fejlesztőkörnyezetből úgy vizsgálhatjuk, mintha a helyi számítógépről futtatnánk azt. A mi alkalmazásunkban ez a program teremt kapcsolatot a telefonon lévő alkalmazással és a számítógéppel. 78

Application Identifier Az OpenPGP Smart Card szabvány [22] által definiált, a kártyára jellemző azonosító bájtsorozat. Lásd az 5.5.2. pontot a pontos felépítéséért. 40, 78

Application Programming Interface Egy (könyvtár-)implementáció más komponensek által meghívható, nyilvánosan elérhető részei. C nyelven íródott könyvtár esetén a nyilvános fejlécállományaiban szereplő függvényeit értjük rajta. 78

Application Protocol Data Unit Application Protocol Data Unit. Egy kommunikációs csomag, ami segítségével az intelligens kártya és a kártyaolvasó kommunikál egymással. 29, 78

Basic Input Output System Az operációs rendszer számára hardveres szolgáltatásokat nyújtó interfész. A BIOS rövidítéssel ennek a rendszernek a beállító programjára is szoktunk hivatkozni. 78

Canonical S-expression Egy S-expression binárisan kódolt formája. 40, 78

Certificate Authority PKI rendszerben szerveződő kulcshierarchiában, végponti kulcsokat hitelesítő (digitálisan aláíró) harmadik szereplő, mely a végpontok bizalmát élvezi. 78

Certificate Revocation List PKI rendszerben szerveződő kulcshierarchiában, azon végponti kulcsok listája, amiről a CA utólagosan visszavonási tanúsítványt adott ki. 78

Chip Card Interface Device Egységes USB kártyaolvasó protokoll, a támogatottsága általában feltételezhető a végponton. 32, 78

CMake Egy modernebb megoldás alacsony szintű szoftverek fordításának szervezésére. Lásd még: GNU Autotools. 47, 51, 68, 71, 72

csomagkezelő Szoftverkomponensek online szerverről való telepítésére, naprakészen tartására és eltávolítására alkalmas szoftvert értünk alatta. Alkalmazása többnyire GNU/Linux rendszereken, illetve fejlesztői eszközök körében elterjedt. 18

Cygwin, vagy MSYS2 / MinGW Mindhárom szoftvercsomag alkalmas rá, hogy Windows operációs rendszer felett Linux eszközök futtatására alkalmassá tegye a környezetet - nem a tárgykód szintjén emuláció, hanem portolás útján. Mindegyikhez tartozik csomagkezelő és `bash` parancsértelmező. A MinGW/MSYS2 környezeteket elsősorban a Linux környezetben elérhető programok átemelésére fejlesztették. Példának okáért a GIMP fénykép manipuláló programot MSYS2 környezetben ültetik át Windows rendszerre. 34

Executable and Linkable Format Gépi kódra lefordított tárgykódot tartalmazó futtatható állományok vagy dinamikusan linkelhető könyvtárak egy lehetséges formátuma. Gyakran találkozhatunk vele Unix-szerű rendszereken. 78

Extended Validation SSL Certificate Jogi személyt azonosító elektronikus tanúsítvány, igazolt nyilvános kulcs. A böngésző címsorában megjelenik a vállalat neve[14]. 78

Extensible Messaging and Presence Protocol Azonnali üzenetküldő (IM, Instant Messaging) protokoll. 78

Filesystem Hierarchy Standard Irányelv Linux disztribúciók számára a fájlrendszer könyvtárstruktúrájáról [5]. 34, 78

Foreign Function Interface Egy programnyelv megoldása egy másik programnyelvben íródott könyvtár használatára. 75, 78

Git Verziókövető rendszer. Elsősorban - de nem kizárólag - szöveges dokumentumok, forráskódok verziózását, változásainak nyomon követését szolgáló program. 31, 77

GNU Autotools A GNU `automake`, `autoconf` programokat, és még néhány hozzájuk tartozó segédprogramot értünk rajta. Együtt úgynevezett szoftverépítő rendszert alkotnak ("software build system"). Feladatuk, hogy lehetővé tegyék a forráskódok hordozását különböző operációs rendszerekre úgy, hogy csak a `bash` parancsértelmezőt, a `make` programot, és a fordító programkészletet feltételezik (amilyen például a GCC). 33, 46, 47, 51, 71, 74

GNU Compiler Collection Eredetileg a GNU operációs rendszer számára írt fordító programcsomag. 78

GNU Privacy Guard A dolgozatban tárgyalt nyílt forrású titkosító szoftver, mely az OpenPGP szabványt teljes egészében megvalósítja. Többnyire GNU/Linux platformon használatos e-mailek hitelesítésére, szoftvercsomagok aláírására, és egyéni felhasználásra. 78

Government Certificate Authority Kormányzati hitelesítés Szolgáltatás[15]. 78

HyperText Transfer Protocol Alkalmazás rétegbeli protokoll, eredetileg egymásra hivatkozó dokumentumok és egyéb médiatartalmak közlésére. Előnye a projektünk tekintetében, hogy megnevezhetőek rajta erőforrások, különböző típusú kérelmek, és az adatok integritását az alatta lévő TCP rétegnek köszönhetően - feltéve, nincsen jelen rosszindulatú támadó - általában garantálja. 78

intelligens kártya Hardveres biztonsági eszköz. Alkalmas arra, hogy kis méretű érzékeny információt eltároljon, és azokkal számításokat végezzen. Ezáltal többek között lehetővé teszi a tulajdonosa digitális azonosítását. Lásd a 3.3.3. pontot. 18

Inter-process communication Egy operációs rendszer két folyamata egymással ahhoz hasonlóan kommunikál, működik együtt, mint ahogyan két különböző számítógépen futó hálózati alkalmazások esetében várnánk el. 78

Java Native Interface Foreign Function Interface (FFI) keretrendszer Java platformból C nyelvben íródott programok függvényeinek hívásához. 43, 78

[14] `https://en.wikipedia.org/wiki/Extended_Validation_Certificate`

[15] `https://nisz.hu/hu/gov-ca-korm%C3%A1nyzati-hiteles%C3%ADt%C3%A9s-szolg%C3%A1ltat%C3%A1s`

kulcstároló Ebben a dolgozatban olyan kis teljesítményű, segédeszközként alkalmazott hardvert (vagy az azon futó szoftvert) értünk rajta, ami képes reprodukálni egy titkosítási kulcsok használatára fenntartott intelligens kártya funkcióit - azaz képes a kulcs kiadása nélkül a kriptográfiai műveletek elvégzésére, az eredmény közlésére, és valamilyen formában a használója azonosítására, valamint a belső állapotának a külvilág elöli elrejtésére. 19, 23–30, 32, 33, 35, 44, 51, 71, 73

KVM Kapcsoló, ami lehetővé teszi, hogy több számítógépet irányítsunk ugyanazzal a billentyűzettel, egérrel és monitorral. 19

multiple-precision integer Tetszőleges aritmetikai pontossággal megvalósított egész szám, melynek esetében annak nagyságát csak a rendelkezésre álló memória korlátozza. 41, 78

Near Field Communication Rövid hatótávú vezeték nélküli kommunikációt megvalósító technológia. Egyes smart kártyák, okostelefonok rendelkeznek a szabványt megvalósító modullal. 78

Off-the-Record Messaging Azonnali üzenetküldő rendszerekben alkalmazható titkosítási protokoll, mely megvalósítja a Perfect Forward Secrecy tulajdonságot. 78

OpenPGP Eredetileg e-mail levelek titkosítására és aláírására kiadott szabvány, amit az RFC 4880 [7] definiál. A Pretty Good Privacy (PGP) rövidítése: Pretty Good Privacy. 19, 75

Personal Computer / Smart Card Egységes kártyaolvasó szabvány. Linux alapú rendszereken megvalósítja a `pcscd`, Microsoft Windows operációs rendszeren a `WinSCard` program. 78

Position-independent code Olyan tárgykódot értünk rajta, ami, miután a memóriába betöltődött, futni képes, a memóriában elfoglalt címére tekintet nélkül. Megosztott (dinamikusan linkelhető) függvénykönyvtárakban alkalmazzák, így más programok általi használatakor ugyanaz a programkönyvtár többször is betölthető különböző memóriaterületekre anélkül, hogy átfedések keletkeznének annak más, használatban lévő területeivel. A használata tehát gyakorlatilag egy további absztrakciót vezet be a címzés során - és ezáltal a program futását valamivel lassítja, kevésbé optimális tárgykódot eredményezve. Nem feltétlenül indokolt minden processzor architektúra esetében, de a biztosabb kompatibilitásért általában elvárt, hogy a dinamikusan linkelhető könyvtárakra jellemző legyen, illetve azokra a statikusan linkelhető könyvtárakra, amiket később fel szeretnénk használni dinamikusan linkelhető könyvtárakban. 50, 78

Pretty Good Privacy Az eredeti, először 1991-ben kiadott titkosító program, melyet Phil Zimmermann írt. 76, 78

Public Key Infrastructure A nyílt kulcsú titkosításban megoldás arra, hogy a terjesztett nyilvános kulcsok eredetiségét bizonyítsuk. Ez egy kulcsaláírás-hierarchiára épül. Például a böngészők ez alapján döntik el, hogy a kapott nyilvános kulcs valóban a kért weboldalhoz tartozik -e[16]. 17, 78

Public-Key Cryptography Standards #11 Hardveres biztonsági modulok, tokenek, intelligens kártyák eléréséhez, kezeléséhez definiált API szabvány. 78

Representational State Transfer (API) Webszolgáltatások fejlesztése során alkalmazott kommunikációs gyakorlatok összessége, amelyben a webes erőforrások állapotait szöveges formában - azon belül gyakran JSON vagy Extensible Markup Language (XML) formátumban - közöljük. 78

[16] `https://en.wikipedia.org/wiki/Public_key_infrastructure`

Run-time search path ELF állományokban előre bekódolt keresési útvonal a dinamikusan betöltendő könyvtárak elérésére. 38, 78

S-expression S-expression, vagy "symbolic expression" alatt egymásba ágyazott lista adatszerkezetek leírására alkalmas jelöléstant értünk. Eredetileg a Lisp programozási nyelv szintaxisaként alkalmazták. 42, 74, 78

Secure Real-time Transport Protocol Valós idejű adatfolyam, többnyire VoIP hívások hanganyagának titkosítására használható protokoll. 78

Service Oriented Architecture Olyan programszervezési architektúra, ahol a szoftvert több különböző, egymástól jól elkülöníthető folyamatra bontjuk - úgynevezett microservice szolgáltatásokat hozunk létre. A feladatot a megoldás a komponensek együttműködésével állítja elő. Előnye az erőforrások könnyebb szétosztása, a könnyebb skálázhatóság, esetenként a nagyobb biztonság, és az egyszerűbb, külön verziózható komponensek fejlesztésének lehetősége. Hátránya, sok esetben lassú vagy nagyobb az erőforrásigénye, illetve, külön problémakört vet fel a komponensek illesztése, kommunikációja. 32, 78

Session Initiation Protocol Két vagy több résztvevő közötti kapcsolat felépítésére használatos kommunikációs protokoll, legelterjedtebben interneten keresztüli telefonhívások (VoIP) felépítésekor alkalmazzák. 78

Software Development Kit Általában egy adott platformra építő fejlesztői eszközök gyűjteményét értjük rajta. 78

status line A GnuPG komponensei által használt Assuan modulok egyik fajta IPC üzenete. 39, 41

Subversion A Git megjelenése előtti egyik legelterjedtebb verziókövető rendszer. 31, 78

syslog Egy 1980-as években, a Sendmail projekt részeként fejlesztett naplózási szabvány, ami idővel a Unix-szerű rendszerek íratlan, központi naplózási megoldásává vált. A C nyelvbeli interfészt általában a `syslog.h` definiálja. Szabványként előnye, hogy a fejlécállomány függvényei operációs rendszer és operációs rendszer között kompatibilis marad, és a naplózás a környezetnek megfelelő helyre lesz irányítva. 39

Transmission Control Protocol Olyan alacsony szintű hálózati réteg, mely az azon közlekedő úgynevezett csomagok ("packet", nem összetévesztendő az APDU csomagokkal) sorrendiségét és hibamentességét ellenőrzi. A kapcsolat megbízhatóságát növeli, de a támadások ellen még nem nyújt önmagában elegendő védelmet. 78

ujjlenyomat Lenyomaton, vagy ujjlenyomaton egy olyan függvénynek az értelmezési tartományát értjük, ami tetszőleges véges hosszú rendezett adathalmazhoz véges, elvárás szerint rövid és változatlan hosszú, rendezett adathalmazt rendel hozzá (általában az utóbbit egy karakterlánc formájában teszik közzé). Elvárás egy ilyen függvénytől, hogy könnyen feldolgozható legyen, míg az inverze viszont rendkívül nehezen. Elvárás, hogy *közelítőleg* kölcsönösen egyértelmű legyen a hozzárendelés - azaz elhanyagolhatón kis valószínűséggel lehessen egy lenyomathoz két különböző bemeneti adathalmazt találni. Használható a bemenet integritásának ellenőrzésére és azonosítására. 23, 40

Web of Trust Elsősorban a PGP titkosító programcsalád megoldása a nyilvános kulcsok hitelességének ellenőrzésének elősegítésére. Kulcshierarchia helyett a felhasználók egymás kulcspárjait írják alá, így lehetővé téve közvetetten egymástól távol eső személyek nyilvános kulcsainak ellenőrzését[17]. 18, 78

ZRTP SRTP protokoll felett alkalmazható peer-to-peer kulcsegyeztető protokoll, melynek során a két fél egy rövid lenyomat hangos beolvasásával győződhet meg a kapcsolat sértetlenségéről. 17

Rövidítések

ADB Android Debug Bridge. 28, 30, 53

AID Application Identifier. 40, 73

APDU Application Protocol Data Unit. 29, 30, 35, 77

API Application Programming Interface. 26, 41, 52, 54, 61, 64, 68, 69, 72, 76

BIOS Basic Input Output System. 20

CA Certificate Authority. 18, 74

CCID Chip Card Interface Device. 32

CRL Certificate Revocation List. 18

csexp Canonical S-expression. 40, 42, 44, 62, 72

ELF Executable and Linkable Format. 38, 51, 58, 77

EV SSL Extended Validation SSL Certificate. 16, 17, 71

FFI Foreign Function Interface. 75

FHS Filesystem Hierarchy Standard. 34, 38

GCC GNU Compiler Collection. 8, 75

GnuPG GNU Privacy Guard. 17, 18, 20, 22, 23, 25, 29–34, 36–40, 42, 46, 56, 57, 66, 69, 71–73, 77

GOV CA Government Certificate Authority. 17

HTTP HyperText Transfer Protocol. 44, 53

IPC Inter-process communication. 31, 77

JNI Java Native Interface. 43, 52, 61, 72

JSON JavaScript Object Notation. 44, 53, 76

MPI multiple-precision integer. 41, 42

NFC Near Field Communication. 33

OTR Off-the-Record Messaging. 17

PC/SC Personal Computer / Smart Card. 32, 33

PGP Pretty Good Privacy. 76

PIC Position-independent code. 50

PKCS#11 Public-Key Cryptography Standards #11. 22, 30

PKI Public Key Infrastructure. 17, 18, 22, 74

REST Representational State Transfer (API). 44, 45, 48, 53–56, 71, 73

rpath Run-time search path. 38

SDK Software Development Kit. 45, 46

sexp S-expression. 42, 43, 71

SIP Session Initiation Protocol. 17

SOA Service Oriented Architecture. 32

SRTP Secure Real-time Transport Protocol. 17

SVN Subversion. 31

TCP Transmission Control Protocol. 53

WoT Web of Trust. 18

XML Extensible Markup Language. 76

XMPP Extensible Messaging and Presence Protocol. 17

[17] `https://en.wikipedia.org/wiki/Web_of_trust`

Hivatkozások

[1] B. Auxier, L. Rainie, M. Anderson, A. Perrin, M. Kumar és E. Turner. (2019. nov. 15.). „Americans and Privacy: Concerned, Confused and Feeling Lack of Control Over Their Personal Information", Pew Research Center, cím: https://www.pewresearch.org/internet/2019/11/15/americans-and-privacy-concerned-confused-and-feeling-lack-of-control-over-their-personal-information/ (elérés dátuma 2020. 04. 13.).

[2] M. Baezner és P. Robin, „Hotspot Analysis: Stuxnet", *Center for Security Studies (CSS), ETH Zürich*, 2018. febr.

[3] P. Bright. (2015. aug. 13.). „Even when told not to, Windows 10 just can't stop talking to Microsoft", cím: https://arstechnica.com/information-technology/2015/08/even-when-told-not-to-windows-10-just-cant-stop-talking-to-microsoft/ (elérés dátuma 2020. 04. 13.).

[4] CardLogix. (2010). „Types of Smart Card", cím: http://www.smartcardbasics.com/smart-card-types.html (elérés dátuma 2020. 04. 22.).

[5] F. Christopher Yeoh; Paul 'Rusty' Russell; Daniel Quinlan; The Linux. (2015. márc. 19.). „Filesystem Hierarchy Standard". 3.0. verzió, LSB Workgroup, The Linux Foundation, cím: https://refspecs.linuxfoundation.org/FHS_3.0/fhs/index.html (elérés dátuma 2020. 10. 12.).

[6] „Constituteproject.org", cím: https://www.constituteproject.org/search?lang=en&q=%22right%20to%20privacy%22 (elérés dátuma 2020. 11. 07.).

[7] L. Donnerhacke, I. GmbH, H. Finney, P. Corporation, D. Shaw és R. Thayer, „OpenPGP Message Format", Standards Track, RFC 4880, 2007. nov.

[8] Felhasználó (m7samuel). (2015. aug. 11.). „What Windows 10 is actually monitoring (regardless of privacy settings)", cím: https://www.reddit.com/r/Windows10/comments/3gm1e3/what_windows_10_is_actually_monitoring_regardless/ (elérés dátuma 2020. 04. 13.).

[9] E. Felten. (2013. okt. 9.). „The Linux Backdoor Attempt of 2003", cím: https://freedom-to-tinker.com/2013/10/09/the-linux-backdoor-attempt-of-2003/ (elérés dátuma 2020. 04. 14.).

[10] „g10 Code", cím: https://g10code.com/p-card.html (elérés dátuma 2020. 10. 10.).

[11] *GnuPG*, 2.2.20. verzió, License: GNU General Public License 3.0.

[12] (2013. ápr. 4.). „ISO/IEC 7816-4:2013 - Identification cards — Integrated circuit cards — Part 4: Organization, security and commands for interchange", International Organization for Standardization.

[13] G. Kelly. (2015. nov. 2.). „Microsoft Admits Windows 10 Automatic Spying Cannot Be Stopped", cím: https://www.forbes.com/sites/gordonkelly/2015/11/02/microsoft-confirms-unstoppable-windows-10-tracking/ (elérés dátuma 2020. 04. 13.).

[14] N. Koblitz, „Elliptic curve cryptosystems", *Mathematics of Computation*, 203–209. old., 48 1987.

[15] J. Kroll. (2013. szept. 20.). „The Debian OpenSSL Bug: Backdoor or Security Accident?", cím: https://freedom-to-tinker.com/2013/09/20/software-transparency-debian-openssl-bug/ (elérés dátuma 2020. 04. 14.).

[16] E. Macaskill és G. Dance. (2013. nov. 1.). „Constituteproject.org", theguardian, cím: https://www.theguardian.com/world/interactive/2013/nov/01/snowden-nsa-files-surveillance-revelations-decoded (elérés dátuma 2020. 11. 07.).

[17] A. Mahdawi. (2019. nov. 5.). „Targeted ads are one of the world's most destructive trends. Here's why". T. Guardian, szerk., cím: https://www.theguardian.com/world/2019/nov/05/targeted-ads-fake-news-clickbait-surveillance-capitalism-data-mining-democracy (elérés dátuma 2020. 11. 08.).

[18] J. Martin. (2016. jún. 8.). „Reviewing Microsoft's Automatic Insertion of Telemetry into C++ Binaries", cím: https://www.infoq.com/news/2016/06/visual-cpp-telemetry/ (elérés dátuma 2020. 04. 14.).

[19] V. S. Miller, „Use of Elliptic Curves in Cryptography", *Lecture Notes in Computer Science*, 417–426. old., 85 1987.

[20] F. Morgner, *Virtual Smart Card*, 2019. okt. 27.

[21] D. D. Patrick Gallagher, „Digital Signature Standard (DSS)", *Federal Information Processing Standards Publication*, 2009. jún.

[22] A. Pietig. (2020. márc. 18.). „OpenPGP Card specification". 3.4.1. verzió, cím: https://gnupg.org/ftp/specs/OpenPGP-smart-card-application-3.4.pdf (elérés dátuma 2020. 10. 14.).

[23] *PKCS #11 v2.11: Cryptographic Token Interface Standard*, Revision 1 verzió, RSA Security Inc. - RSA Laboratories, 2001. nov.

[24] R. Rivest, A. Shamir és L. Adleman, „A Method for Obtaining Digital Signatures and Public-Key Cryptosystems", *Communications of the ACM*, 21. évf., 2. sz., 120–126. old., 1978. febr.

[25] J. Sneddon. (2013. nov. 20.). „NSA Wanted Backdoor Access In Linux, Says Linus Torvalds' Father", cím: https://www.omgubuntu.co.uk/2013/11/nsa-ask-linus-torvalds-include-backdoors-linux-father-says-yes (elérés dátuma 2020. 11. 07.).

[26] (2018. márc. 6.). „Subkeys", cím: https://wiki.debian.org/Subkeys (elérés dátuma 2020. 10. 09.).

[27] *The Libgcrypt Reference Manual*, 1.8.5. verzió, Free Software Foundation, Inc., g10 Code GmbH, 2018. nov. 19.

[28] K. Thompson, „Reflections on Trusting Trust", *Communications of the ACM*, 27. évf., 8. sz., 761–763. old., 1984. aug.

[29] *Using the GNU Privacy Guard*, 2.2.19. verzió, Free Software Foundation, Inc., Werner Koch., g10 Code GmbH., 2019. nov.

[30] A. Waksman és S. Sethumadhavan, „Tamper Evident Microprocessors", *Columbia University*, 2010.